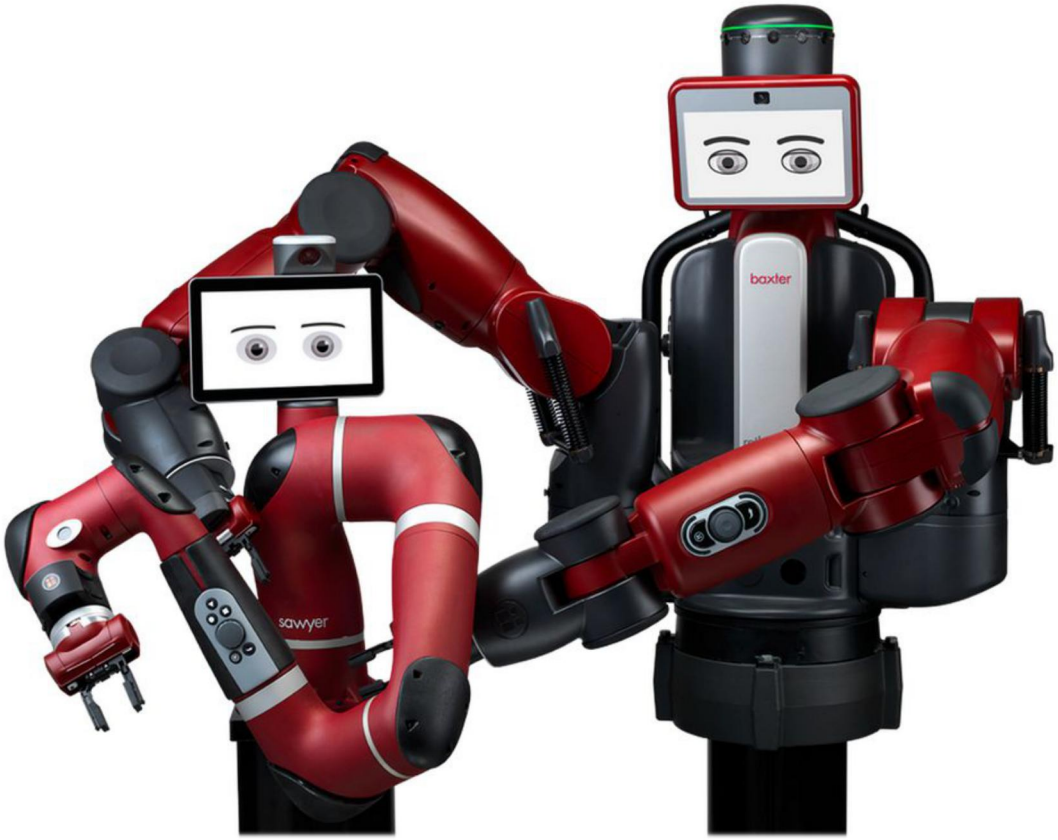
瑞森可机器人实训课程



湖南瑞森可机器人科技有限公司

地址：　长沙市岳麓区桐梓坡西路 348 号科力远园区
国家工程中心二楼
电话：　0731-89872400

瑞森可

前言

机器人本体涉及多门学科交叉融合，从零开始组装涉及一台实验机器人需要耗费大量时间和精力，这对于需要机器人进行开发实践的，如人工智能等专业的同学非常不友好。Ros 机器人系统旨在解决这个问题，且目前 ros 已成为机器人开发的事实标准。

本实训课程旨在通过一系列单个实验，让同学们逐步掌握机器人 ros 开发技能，最终实现包含机器视觉、机械臂运动控制，运动规划等集成功能的机器人视觉伺服抓取项目。机器人视觉伺服抓取作为工业上最为常见的应用需求，同学们掌握后即具备基本的机器人项目开发能力。根据同学们学习曲线的不同，后续同学们还可在该项目中加入如机器学习，深度视觉，语音识别等 Ai 扩展能力。

实训课内容如下：

序号	实验名称	内容说明
1	机器人认知和操作	机械臂关节结构，机械臂基础操作
2	ROS 使用	ROS 基本使用
3	ROS 工具使用	ROS 工具应用
4	Gazebo 仿真	机器人仿真环境使用
5	Moveit 运动规划	机械臂运动规划实现
6	Moveit 环境感知	配合 kinect 环境感知
7	Rviz 使用	Rviz 数据可视化
8	机械臂写字	控制机械臂末端写字
9	运动控制	控制机械臂关节运动
10	视觉图像传递	视觉图像处理

目 录

实验 1 机器人认知与操作.....1
实验 2 ROS 基础.....6
实验 3 ROS 工具的使用.....12
实验 4 机器人仿真实验.....20
实验 5 机器人运动规划实验.....26
实验 6 MoveIt 环境感知实验.....33
实验 7 RVIZ 使用.....38
实验 8 dobot 写字实验.....43
实验 9 机器人控制.....49
实验 10 视觉图像传递.....54

实验1 机器人认知与操作

一、实验目的

1. 感知机器人手臂自由度的概念
2. 了解机器人关节结构
3. 熟悉机器人拖拽示教功能的使用

二、实验所用单元

Baxter 机器人一台

三、实验内容

1. 认识机器人的自由度
2. 根据机器人核心硬件结构示意图认识机器人 SEA 关节与传统刚性关节的区别。
3. 根据实验步骤熟悉机器人的拖拽示教功能

四、实验方法和步骤

机器人自由度

市场目前应用最多的工业机器人，最多是六轴的，此外四轴，五轴的工业机器人也不少，但机械臂既然主要是为了代替人类从事重复，繁重，危险的工作，那么人的手臂有几个自由度？

请大家数一下自己手臂的自由度（手指不算），并填入下表：

自由度数	所在手臂关节及运动范围
第 1 自由度	
第 2 自由度	
第 3 自由度	
第 4 自由度	
第 5 自由度	
第 6 自由度	
第 7 自由度	

答案如下图：



图 1： 人体手臂自由度

然而为什么人手臂是 7 个自由度，而不是 8 个也不是 6 个？大家可以尝试做做平时生活中的一些动作，比如拧钥匙，并保持手腕不动看看完成这个动作的效果和平时的区别。

Baxter 机器人是一款创新的智能协作机器人，有着两只长手臂，每只手臂的 7 个自由度是对人手臂的真实还原。

Baxter 手臂分 3 个部分：肩（S=Shoulder）、臂（E=Elbow）、腕（W=Wrist）。

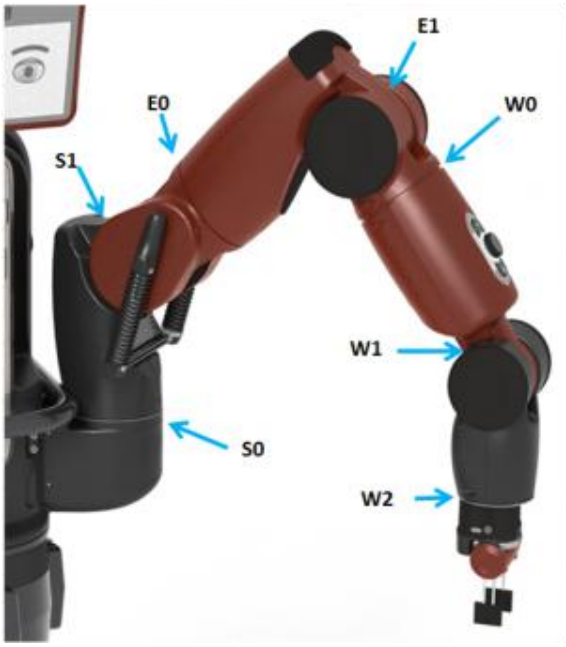


图 2： Baxter 手臂

机器人手臂的关节

长期以来工业机器人的机械臂的旋转关节普遍采用电机输出轴与关节机构进行刚性连接的驱动方式，其主要优势是机构中电机控制的精确性和运动响应的快速性，且技术成熟，配套产业完善。但刚性驱动器的强鲁棒性和弱适应性对于机器人实现如柔顺化，自调整接触等目标任务时存在不足。

相对于传统关节机器人的驱动系统，人体肌肉-骨骼运动系统具有非常大的优势。Baxter 的 SEA 关节结构是利用弹簧特性拟人化运动的一种驱动形式，其原理图与实物结构图如下：

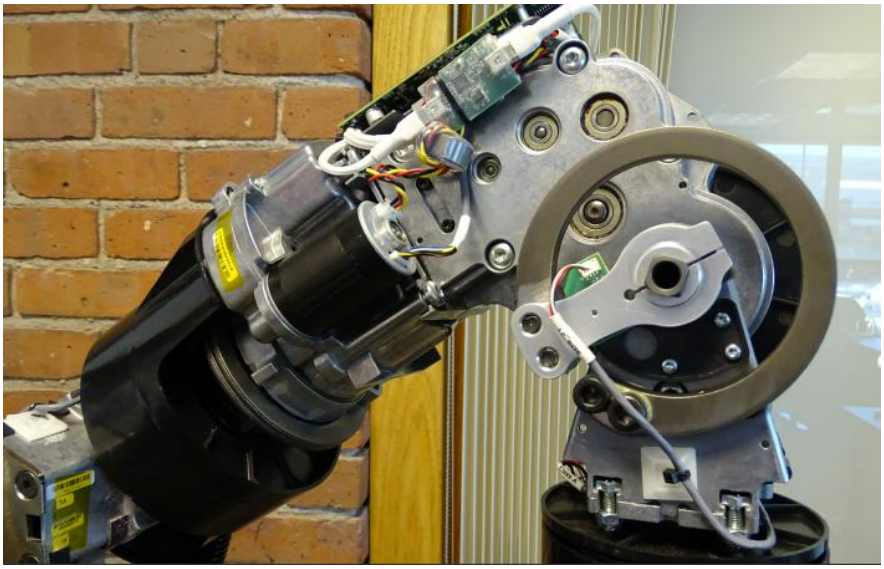


图 3： 手臂控制单元图

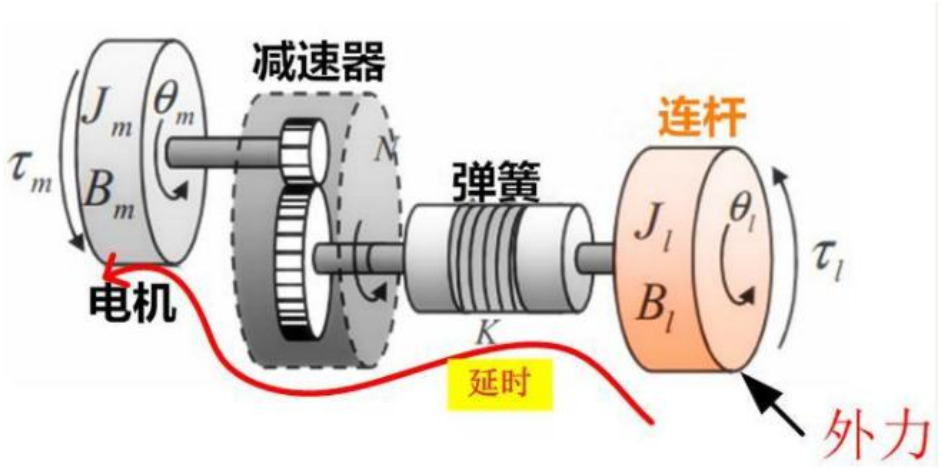


图 4： 串联弹性致动器

通过在机器人每个关节上添加 SEA，可以进行各个关节的扭矩感应，通过电机控制能更有效地控制关节扭矩，同时在冲击和碰撞时可以进行稳定的控制，从而使机器人增加柔性。

注：Baxter 手臂关节均无刹车机构，在没有上电的情况下各关节均可以手动移动。请尝试在上电情况下推动 Baxter 手臂管关节并观察 Baxter 手臂关节的弹性幅度。

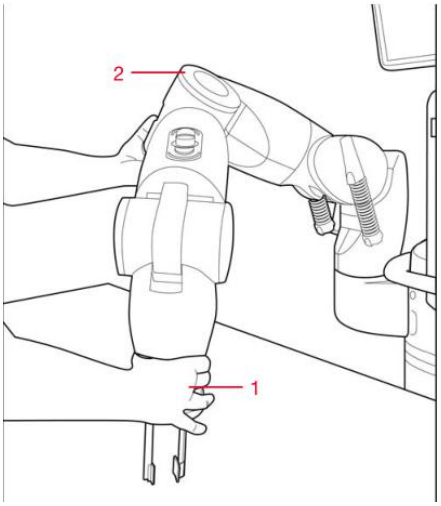
问题：Baxter 手臂的活动范围是否基本符合人的手臂的活动范围？

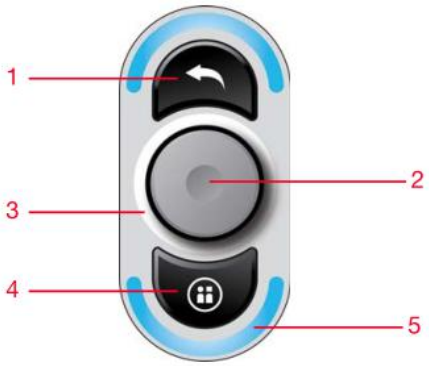
拖拽示教功能

目前，机器人的示教工作主要是依赖于工人操作示教器来完成的。然而，这种方式的缺

陷很多，如示教效率低，示教过程复杂，并且需要操作员掌握一定的机器人使用知识。

机器人拖动示教，就是操作员可以直接拖着机器人各关节，运动到理想的姿态，记录下来。协作机器人是较早具有该功能的系统。





1. 返回键 2.确定键 3. 滚轮选择旋钮 4.菜单键 5 触摸感应指示灯

目前拖动示教可以分为两大方向：增加传感器类和运用物理学公式类

传感器类：是在机器人关节处安装力矩传感器，配合控制器中的算法，操作者牵引机器人末端执行器做线性或者旋转的运动，检测出用户施加在该处的外力信息的拖动示教。

物理学公式类：这种方式不需要在机器人本体上加其余力矩传感器，优势是成本低，结构简单，但手感会比传感器类稍微差点。这种类型的拖动示教，机器人驱动器工作在电流/力矩模式。

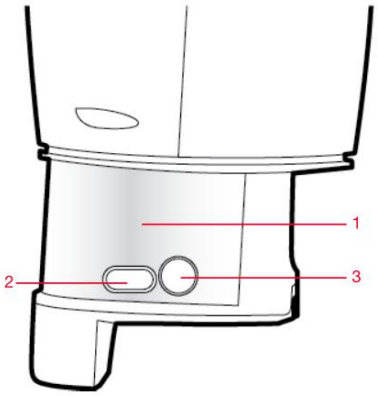
Baxter 机器人的拖动示教方法是采用在关节处安装力矩传感器和双编码器， 组成柔性关节或线弹性驱动器（serial elastic joint）。由于关节刚度较低，电机与连杆端的角度不一样，因此需要在连杆端（力矩传感器输出） 再加一个编码器。这种结构的优势是可以精确对系统建模， 提高力矩检测和控制精度，拖动示教也会更轻松更柔顺。

拖拽示教方法：

1 使用 Baxter 手臂上的导航旋钮选择例程，使用旋钮单击选择 Record



2.按住 Baxter 手臂上的触摸按钮直到导航按键上的蓝色指示灯亮起，这时可自由拖动 Baxter 手臂，Baxter 机器人将同时记录拖拽示教过程中的位置变化，速度变化和时间戳。



1.触摸开关按钮 2.夹爪开关按钮 3.确认键

3.示教结束后使用导航键旋钮选择返回，然后选择 play，Baxter 将开始回放刚才示教的动作。



五、实验要求

本实验要求完成以下内容：

1. 理解不同自由度的机械结构对机械臂的运动范围的影响
2. 阐述 Baxter 机器人 SEA 关节结构原理
3. 使用拖动示教完成一个在键盘上打字的任务

六、注意事项

本实验课为公共科目，由于设备数量有限，请同学们听从实验老师的指导。注意课堂纪律、遵守实验室规章制度和设备使用规范。爱护爱惜实验设备。

实验2 ROS 基础

一、 实验目的

1.了解 ROS 的用途

2. 学习 ros 基本命令

3.学习机器人基本例程的使用

二、 实验所用单元

Baxter 机器人一台；

电脑一台（内存 4G 以上、至少 20G 可用硬盘空间）

路由器一台

网线 2 条。

三、实验内容

1.了解 ros 的应用现状和基本特点

2.ros 系统入门使用

3. 根据实验步骤控制机器人手臂动作

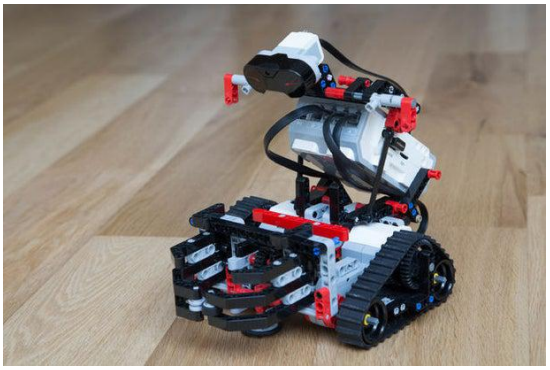
四、实验方法和步骤

为什么使用 ROS（Robot Operating System）

ROS 原本是斯坦福大学的 PR2 机器人项目，后来由 Willow Garage 公司开源并推动其发展，目前是由 OSRF（Open Source Robotics Foundation, Inc）公司维护的开源项目。ROS 的出现解决了对机器人的硬件进行了封装的需求。不同的机器人、不同的传感器，在 ROS 里可以用相同的方式表示（topic 等），供上层应用程序（运动规划等）调用，这大大提高了机器人的开发效率。而随着机器人领域的快速发展和复杂化，代码的复用性和模块化的需求越来越强烈，机器人研究领域很快开始了学习和使用 ROS 的热潮。

ROS 目前已广泛应用于 Clearpath 物流机器人、Fetch 导购机器人、DJI 大疆无人机、Apollo 自动驾驶汽车、Nao 舞蹈机器人、Lego 玩具机器人、iRobot 扫地机器人、Pepper 情感机器人、Rethink 协作机器人等。









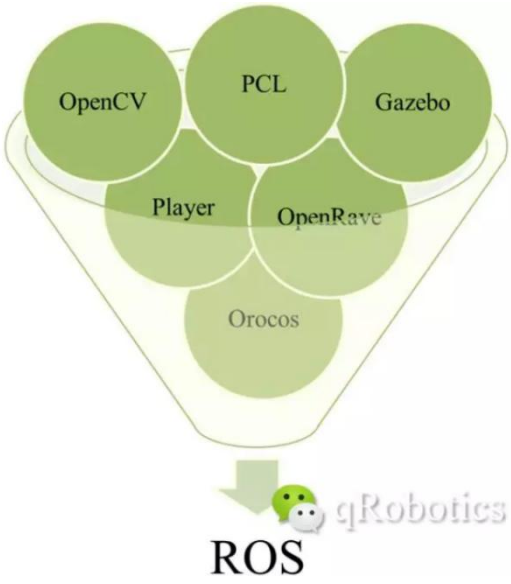
ROS 的主要特点：

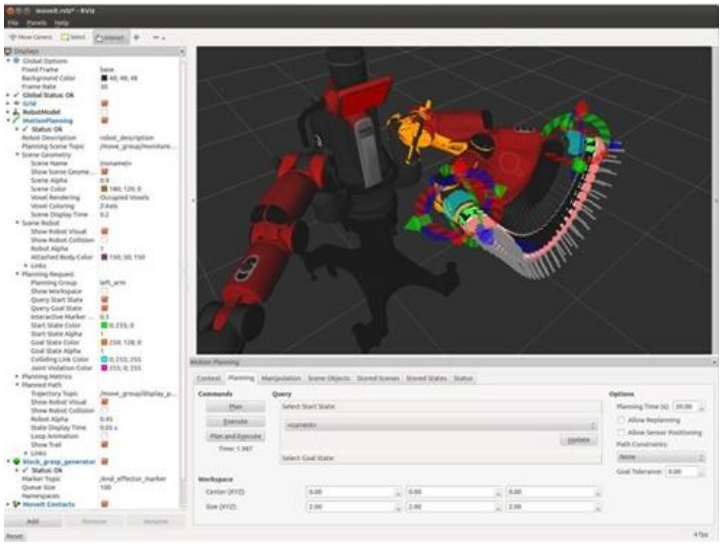
①精简与集成：ROS 建立的系统具有模块化的特点，各模块中代码可以单独编译，而且编译使用的 Cmake 工具可以很容易的实现精简的理念。ROS 集成了很多现在已经存在的开源库，例如：OpenCV、OpenRAVE、Player 等。

②支持多种编程语言：为满足不同编程者的需求，ROS 采用了语音中立性的框架结构，即不限某一种编程语音。ROS 支持语音包括 C++、Python、Octave 和 LISP 等。

③免费并且开源: ROS 遵循 BSD 协议,对个人、商业应用以及修改都是免费的。

④点对点设计:ROS 的点对点设计以及服务和节点管理器等机制可以分散由计算机视觉和语音识别等功能带来的实时计算压力。





我们使用 turtlesim 运行的例子来学习 ros 基本使用：

启用 turtlesim 在三个不同的终端中，分别执行以下三个指令：

①、第一个终端中输入如下指令，启动 roscore 运行：

$ roscore

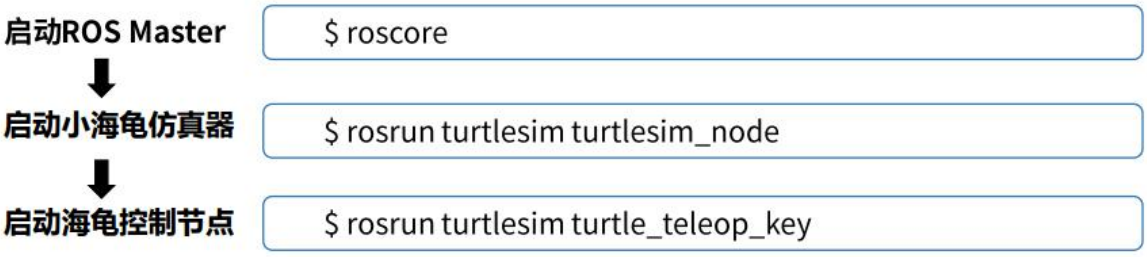
②、第二个终端中输入如下指令，启动小海龟仿真节点：

$ rosrun turtlesim turtlesim_node

③、第三个终端中输入如下指令，启动键盘控制节点：

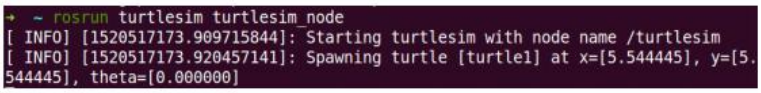
$ rosrun turtlesim turtle_teleop_key

④、如下图所示：





小海龟仿真器界面



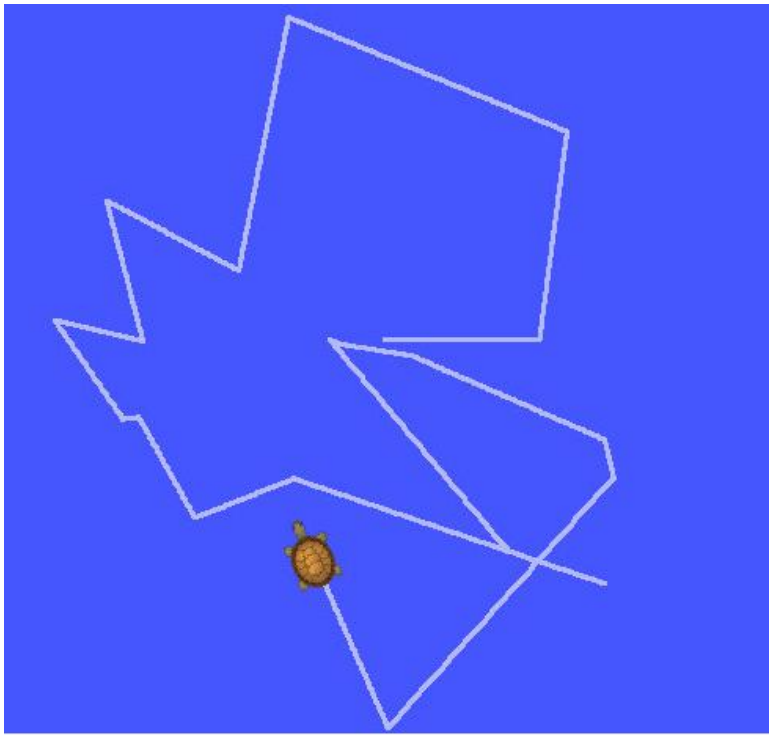
启动海龟仿真器节点



启动海龟控制节点

https://blog.csdn.net/qq_40213457

⑤、切换到第三个终端（执行命令为 turtle_teleop_key），然后按键盘上、 下、 左、或右键，海龟就会响应你的命令开始移动了，并在屏幕上留下运动轨迹。

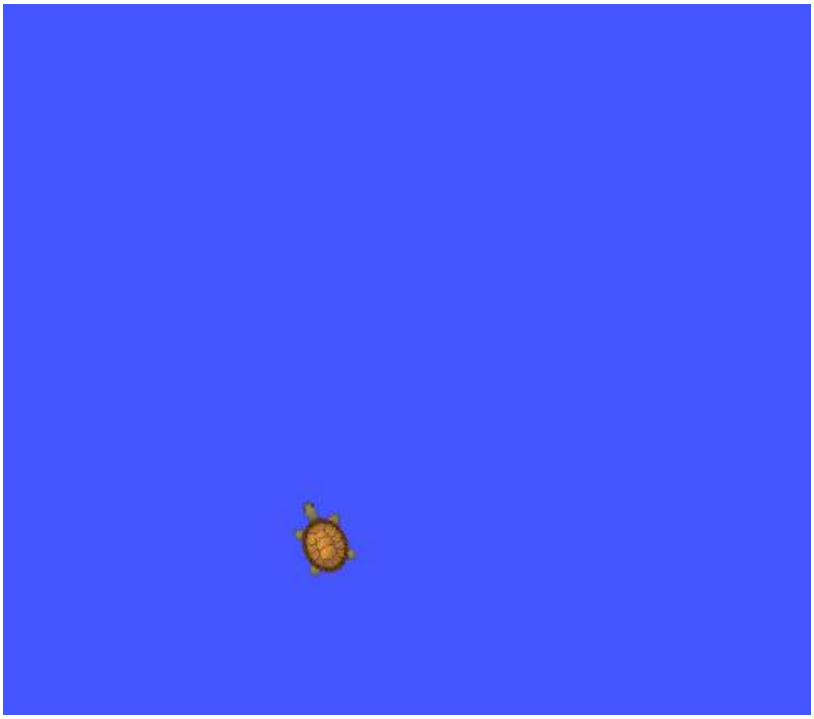


ROS 命令释义

roscore	启动ros 主节点及参数服务等
rosrun	运行可执行文件
roslaunch	启动launch 文件（包含多个可执行文件）

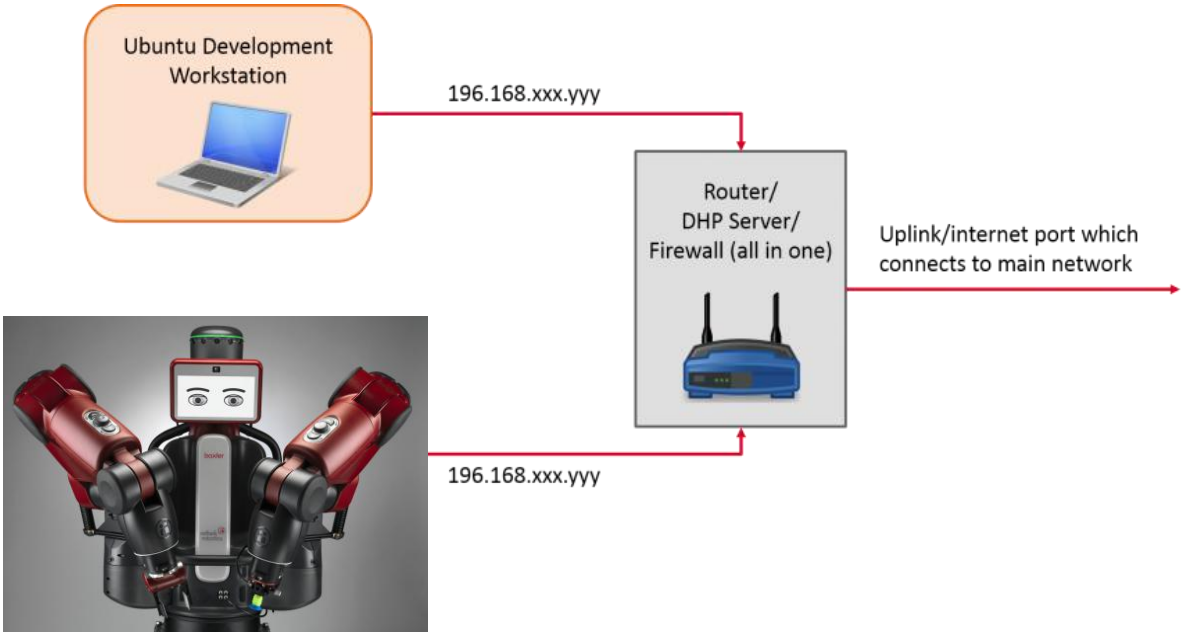
调用 $ rosservice call clear

得到结果：



使用 ros 控制实体机器人 Baxter

如下所示，将 Baxter 机器人和电脑通过网线链接的方式链接到路由器局域网上。



打开新的终端，执行命令时，应该首先按下面步骤初始化 Baxter 机器人 SDK 运行环境

$cd ~/ros_ws

$./baxter.sh

使能机器人：

1)、案例运行是需要先使能机器人，使能 Baxter 机器人命令如下：

$ cd ~/baxter_ws（其中 baxter_ws 文件为自己在机器个创建的 SDK 工作空间存放的文件）

$ ./baxter.sh

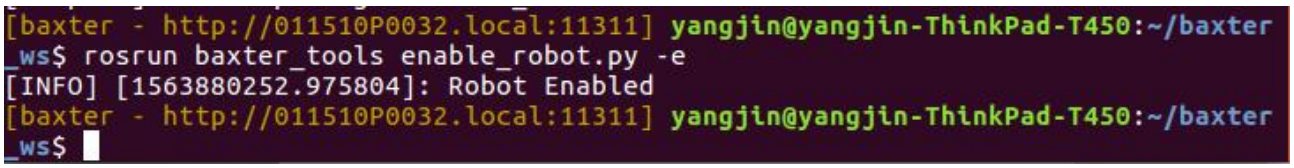
2)、再 ping 一下机器人编码，看是否可以 ping 通成功，ping 通结果如 上所示，说明电脑和 Baxter 机器人连接了,可以开始运行案例。

例如：$ ping 011510P0032.local

3)、启用机器人命令：

$ rosrun baxter_tools enable_robot.py –e

4)、终端输出界面图如下，如果输出的结果显示 Robot Enabled 表示已经使动 Baxter 机器人



键盘操控关节运动例程调用

⑵、键盘操控关节运动：

介绍：开发机器人人员用键盘来控制 Baxter 手臂的每个关节位置与夹爪的开合。

操作方法：

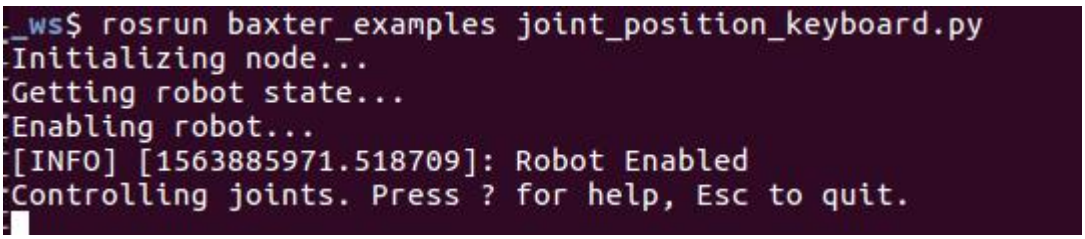
1) 、验证是否已启用机器人

$ rosrun baxter_tools enable_robot.py -e

2)、案例运行使用命：

$ rosrun baxter_examples joint_position_keyboard.py

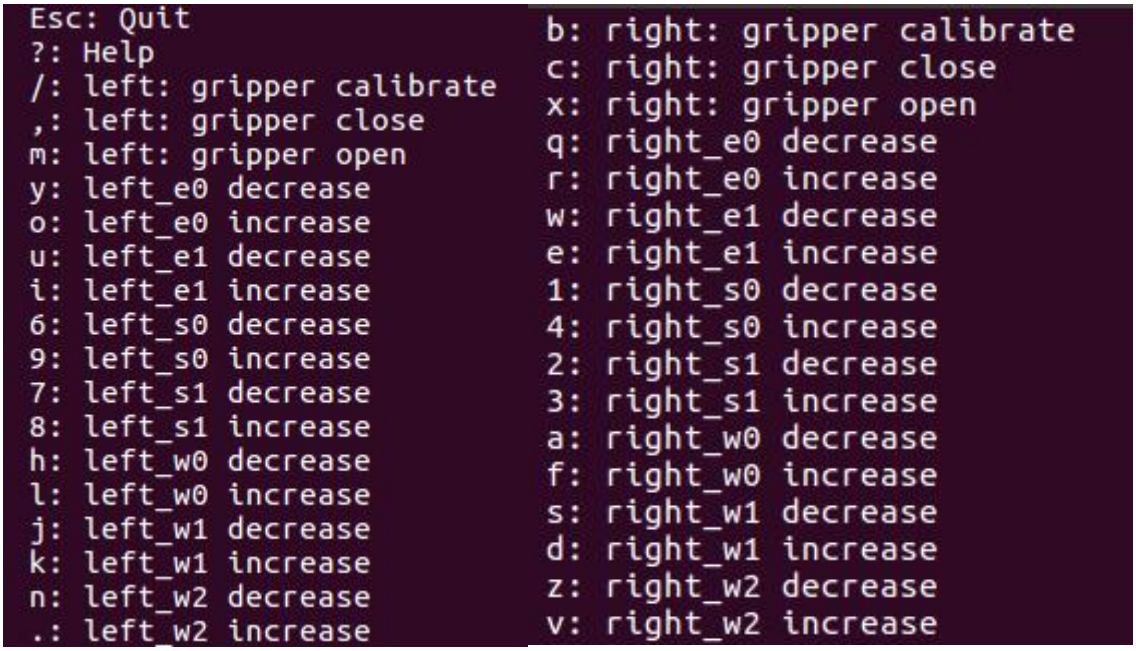
通过键盘控制 Baxter 关节点的运动,键盘上的数字、字母、符号分别控制关节角度的增加、减少以及夹角的开合与校准。如下图所示：



可选参数：

-h, --help --可选,帮助信息

输入节点命令后,按下 shift+?号出现键盘控制提示



参考网址：

例程：http://sdk.rethinkrobotics.com/wiki/Joint_Position_Keyboard_Example

源码：http://sdk.rethinkrobotics.com/wiki/Joint_Position_Keyboard_-_Code_Walkthrough

四、实验报告

本实验要求完成以下内容：

1. 了解 ROS 系统的用途
2. 学习使用 ROS 基本命令
3. 操作 Baxter 键盘控制例程抓取并放置物品

五、注意事项

本实验课为公共科目，由于设备数量有限，请同学们听从实验老师的指导。注意课堂纪律、遵守实验室规章制度和设备使用规范。爱护爱惜实验设备。

实验3 ROS 工具的使用

一、 实验目的

1.了解 ROS 系统基本原理

2.学习 ROS 应用操作

3.学习机器人基本例程的使用

二、实验所用单元

Baxter 机器人一台；

电脑一台（内存 4G 以上、至少 20G 可用硬盘空间）

路由器一台

网线 2 条。

三、实验内容

1.了解 ros 系统的基本概念

2. ros 命令和工具的使用

3. 根据实验步骤使用关节弹簧例程

四、实验方法和步骤

1、ROS 工具使用

关于 ros 几个重要概念

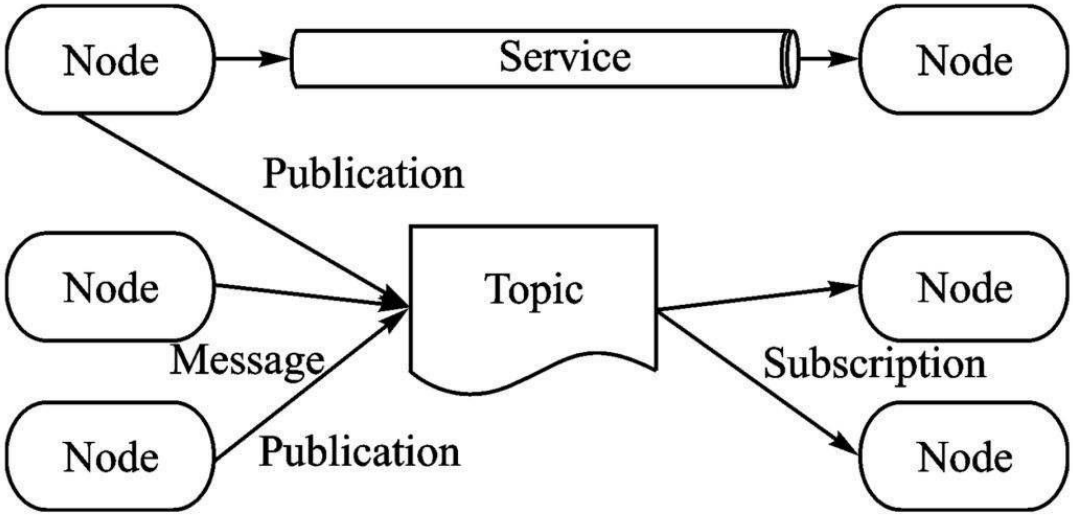
节点（Node）：节点是用 C++或 Python（ROS 客户端库 roscpp、rospy）编写的程序，是主要的计算执行进程。如果你想要有一个可以与其他节点进行交互的进程，那么你需要创建一个节点，并将此节点连接到 ROS 网络。。

消息（Message）：节点之间通过消息进行通信，这些消息包含一个节点发送给其他节点的信息数据，消息类型有 ROS 标准类型和基于标准消息开发的自定义类型两种。

主题（Topic）：主题是由 ROS 网络对消息进行路由和消息管理的数据总线。每一条消息都要发布到相应的主题。当一个节点发送数据时，我们就说该节点正在向主题发布消息。节点可以通过订阅某个主题，接收来自其他节点的消息。一个节点可以订阅一个主题，而并不需要该节点同时发布该主题。这就保证了消息的发布者和订阅者之间相互解耦，完全无需知晓对方的存在。

服务（Service） 在发布主题时，正在发送的数据能够以多对多的方式交互。但当你需要从某个节点获得一个请求或应答时，就不能通过主题来实现了。在这种情况下，服务能够允许我们直接与某个节点进行交互。此外，服务必须有一个唯一的名称。当一个节点提供某个服务时，所有的节点都可以通过使用 ROS 客户端库所编写的代码与它通信。

它们之间关系图如下：



我们使用 turtlesim 运行的例子来学习 ros 工具使用。启用 turtlesim 在三个不同的终端中，分别输入以下指令：

1. $ roscore
2. $ rosrun turtlesim turtlesim_node
3. $ rosrun turtlesim turtle_teleop_key

Ros 基本命令

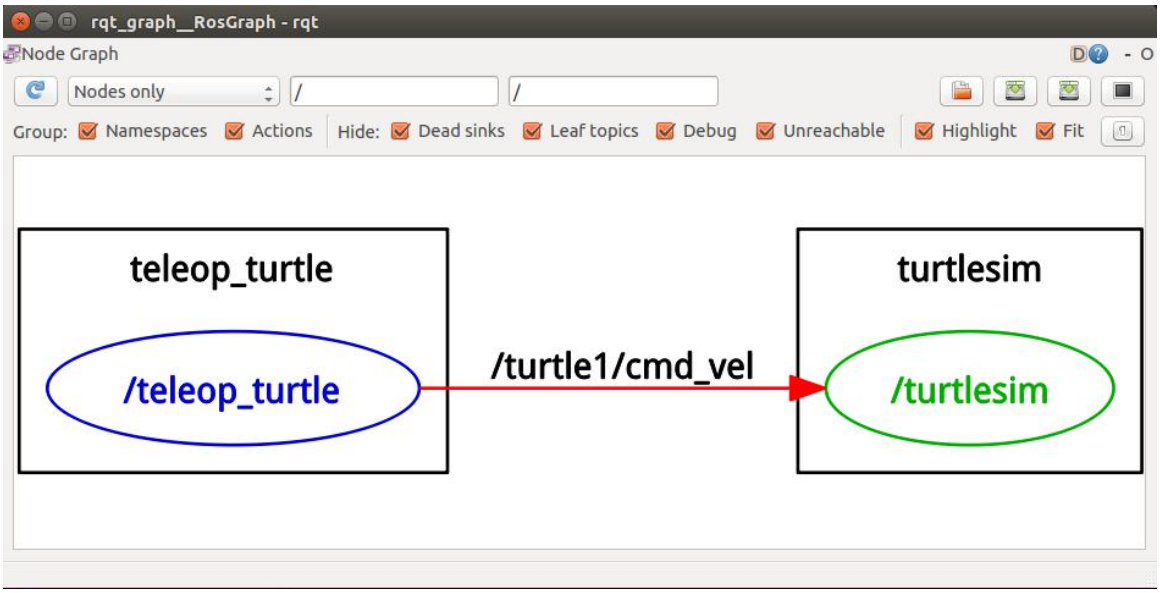
rosmsg show	显示消息定义
rosmsg list	显示全部消息名字
rosnode list	列出所有活动节点
rosnode info	输出节点信息
rosnode machine	列出某机器人上运行的节点
rosnode kill	终止一个节点
rostopic bw	topic 带宽
rostopic echo	显示 topic 内容
rostopic hz	topic 发布频率
rostopic info	topic 信息
rostopic list	所有 topic 列表
rostopic pub	向 topic 发布消息
rostopic type	topic 类型
图形界面工具	
rqt_console	显示 rosout 上的消息

rqt_bag	记录、回放 bag 文件
rqt_topic	查看发布的 topics
rqt_msg/srv	显示消息或服务定义
rqt_publisher	向 topic 发送消息
rqt_graph	节点关联图
rqt_dep	软件包依赖关系
rqt_reconfigure	参数动态配置
tf_echo	打印坐标系间的坐标变换
view_frames	查看系统坐标变换树
rqt_plot	画出 topic 数据图形

1.第二个终端中输入如下指令，使用 ros 工具：

$rqt_graph

rqt_graph 工具将打开小海龟例程的节点关联图。如下，蓝色和绿色为节点，中间横线上的/turtle1/cmd_vel 是主题。



在 turtlesim 的例子中，我们创建了两个节点。第一个节点是可执行文件 turtlesim_node 的实例化。这个节点负责创建 turtlesim 窗口和模拟海龟的运动。第二节点是可执行文件 turtle_teleop_key 的实例化。teleop 是 teleoperation（遥操作）的缩写，是指人通过在远程发送运动指令控制机器人。这个节点的作用是捕捉方向键被按下的事件，并将方向键的按键信息转换为运动指令，然后将命令发送到 turtlesim_node 节点。

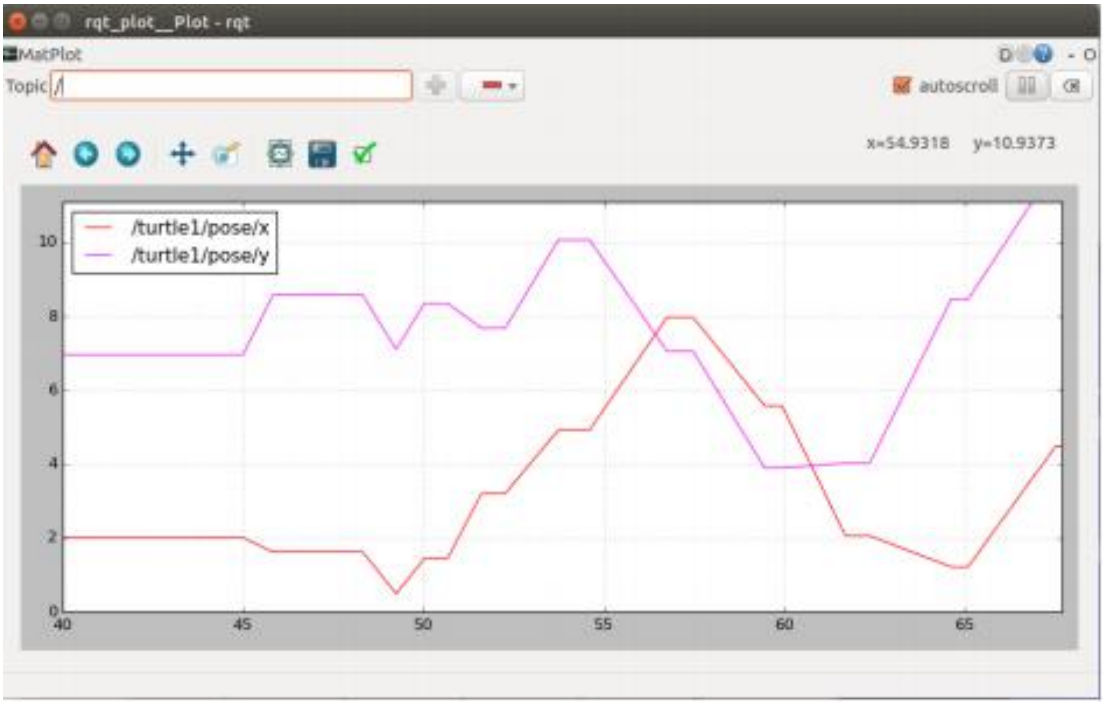
2.第新终端中输入如下指令，使用 ros 工具：

$rqt_plot

在列表框选择（或写入）要监测的话题，例如：/turtle1/pose/（话题的所有参数）或者

/turtle1/pose/x （话题的某一参数），然后点击加号。

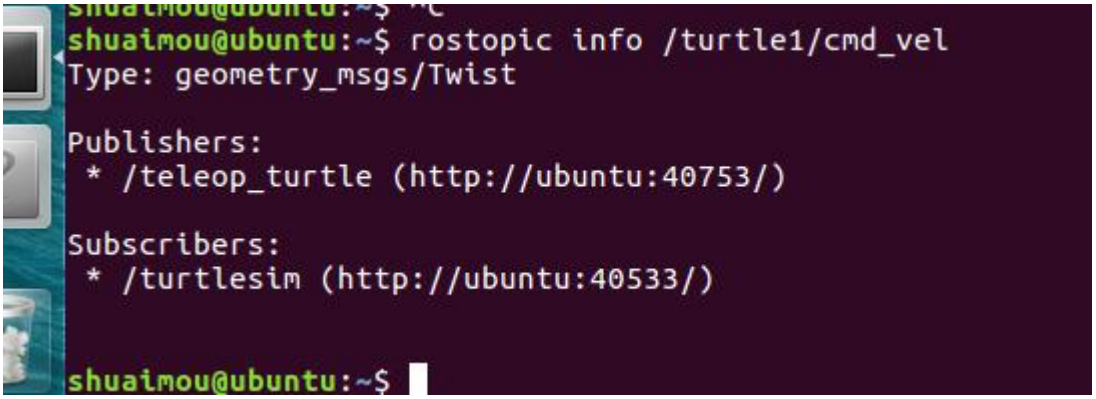
这时，当你使用键盘控制小海龟移动时，rqt_graph 将对/turtle1/pose/x 的数据进行绘图显示，这样可以更加直观看到话题消息的变化，方便于我们调试。



订阅某个话题，如：

rostopic info /turtle/cmd_vel

可以看到发布者节点，接受者节点



监听控制小乌龟运动的节点：

rostopic echo /turtle/cmd_vel

移动海龟，命令窗口里就可以看到键盘发出的指令信息。

linear 为线速度，angular 为角速度，单位是 m/s 和 rad/s。线速度 linear，x 是前后。角速度 angular， z 是沿 Z 轴逆时针转动。

```
shuaimou@ubuntu:~$ rostopic echo /turtle1/cmd_vel
linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: -2.0
```

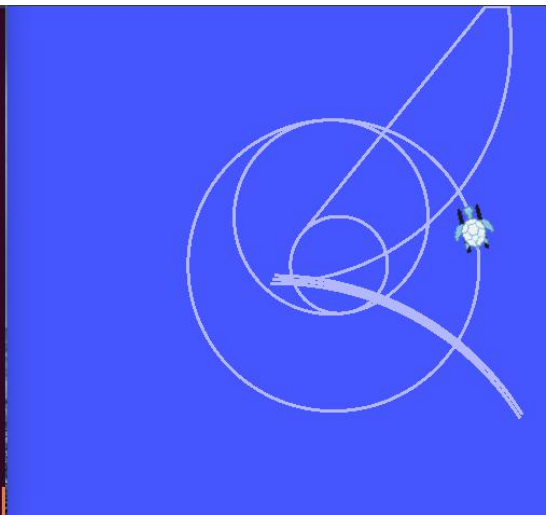
通过 rostopic pub 命令，给指定的话题发布消息

rostopic pub -r 10 /turtle/cmd_vel geometry_msg/Twist +table 键

/turtle1/cmd_vel 为话题，geometry_msgs/Twist 为数据类型。

改变参数后按 enter，小海龟即开始按照发布的指令运动。

```
r:
  x: 1.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 1.0"
peng@peng-E460:~$ rostopic pub -r 10 /turtle1/cmd_vel geom
etry_msgs/Twist "linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 1.0"
^Cpeng@peng-E460:~$ rostopic pub -r 10 /turtle1/cmd_vel ge
etry_msgs/Twist "linear:
  x: 3.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 1.0"
```



通过 rosbag record/play 命令，记录指定话题发布的消息

话题记录命令：rosbag recode topic1 topic2

话题信息回放：rosbag play date.bag

在 teleop 节点终端窗口中通过方向键控制海龟移动，后台会自动记录运动状态，并将其记录到 home/date.bag 文件中，Ctrl+c 结束记录。

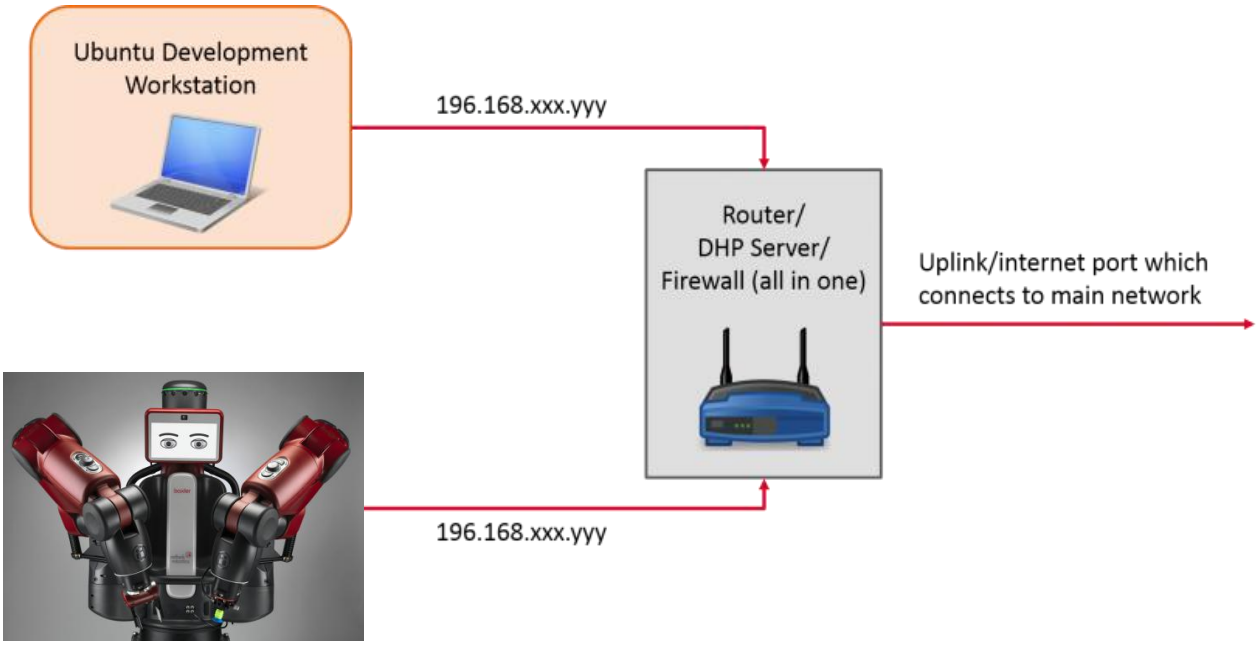
```
[ INFO] [1565766419.674741226]: Recording to 2019-08-14-15-06-59.bag.
^Cpeng@peng-E460:~$ rosbag play 2019-08-14-15-06-59.bag
[ INFO] [1565766452.981201753]: Opening 2019-08-14-15-06-59.bag

Waiting 0.2 seconds after advertising topics... done.

Hit space to toggle paused, or 's' to step.
 [RUNNING]  Bag Time: 1565766424.252084   Duration: 4.336245 / 4.367936
Done.
peng@peng-E460:~$
```

3、使用 ros 控制实体机器人 Baxter

如下所示，将 Baxter 机器人和电脑通过网线链接的方式链接到路由器局域网上。



打开新的终端，执行命令时，应该首先按下面步骤初始化 Baxter 机器人 SDK 运行环境

$cd ~/ros_ws

$./baxter.sh

使能机器人：

1)、案例运行是需要先使能机器人，使能 Baxter 机器人命令如下：

$ cd ~/baxter_ws（其中 baxter_ws 文件为自己在机器个创建的 SDK 工作空间存放的文件）

$ ./baxter.sh

2)、再 ping 一下机器人编码，看是否可以 ping 通成功，ping 通结果如 上所示，说明电脑和 Baxter 机器人连接了,可以开始运行案例。

例如：$ ping 011510P0032.local

3)、启用机器人命令：

$ rosrun baxter_tools enable_robot.py –e

4)、终端输出界面图如下，如果输出的结果显示 Robot Enabled 表示已经使动 Baxter 机器人

```
[baxter - http://011510P0032.local:11311] yangjin@yangjin-ThinkPad-T450:~/baxter
_ws$ rosrun baxter_tools enable_robot.py -e
[INFO] [1563880252.975804]: Robot Enabled
[baxter - http://011510P0032.local:11311] yangjin@yangjin-ThinkPad-T450:~/baxter
_ws$
```

关节弹簧模拟例程调用

⑵、关节弹簧模拟运动：

介绍：这个例子显示了关节扭矩控制的用法。机器人手臂移动到指定位置后，机器人将进入力矩控制模式，施加力矩表示虚拟弹簧，将关节保持在起始位置。

操作方法：

1) 、启用机器人手臂

$ rosrun baxter_tools enable_robot.py -e

2）、案例运行使用命：

$ rosrun baxter_examples joint_torque_springs.py -l left

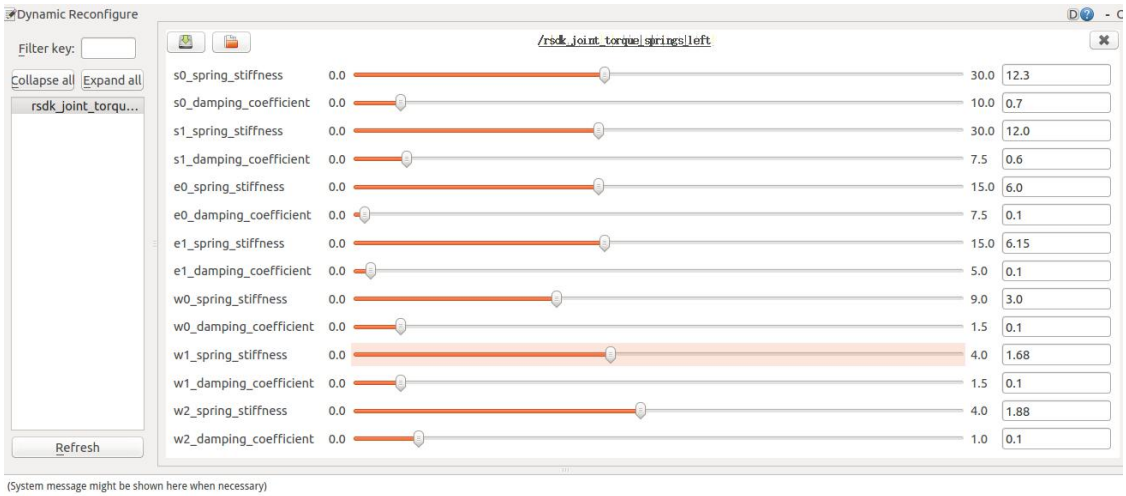
```
[baxter - http://011510P0032.local:11311] peng@peng-E460:~/ros_ws$ rosrun baxter
_examples joint_torque_springs.py -l left
Initializing node...
Getting robot state...
Enabling robot...
[INFO] [WallTime: 1565767504.264005] Robot Enabled
Running. Ctrl-c to quit
```

此时，用手推动 baxter 的左手臂后松开，Baxter 的左手臂的各个关节将会像弹簧一样开始摆动并最终慢慢停止，恢复到初始位置。

Baxter 手臂的控制方式有三种:关节位置控制、关节速度控制和关节扭矩控制。这个例子计算并应用扭矩与起始位置的偏移量线性相关，从而产生了一个错觉，即每个关节上都附加了可配置刚度的弹簧。实例演示了干净利落的转矩输出控制方式。

此外，还可以通过 rqt_reconfigure 实时控制配置各个关节的刚度系数和阻尼系数。

$ rosrun rqt_reconfigure rqt_reconfigure



参考网址：

例程：http://sdk.rethinkrobotics.com/wiki/Joint_Torque_Springs_Example

源码：http://sdk.rethinkrobotics.com/wiki/Joint_Torque_Springs_-_Code_Walkthrough

四、实验报告

本实验要求完成以下内容：

1. 阐述 ROS 系统的基本架构
2. 熟练使用 ROS 命令操作
3. 根据 Baxter 弹簧模拟例程记录关节力矩话题的信息并在 rqt_plot 中回放。

五、注意事项

本实验课为公共科目，由于设备数量有限，请同学们听从实验老师的指导。注意课堂纪律、遵守实验室规章制度和设备使用规范。爱护爱惜实验设备。

实验4 机器人仿真实验

一、实验目的

1. 了解 Gazebo 的安装与基本使用
2. 使用 Gazebo 对机器人控制进行仿真学习

二、实验所用单元

实验电脑一台（安装 Ubuntu16.04 系统、ROS kinetic 系统,可连接互联网）

三、实验内容

3.1 gazebo 简介

Gazebo 是一款 3D 动态模拟器，能够在复杂的室内和室外环境中准确有效地模拟机器人群。与游戏引擎提供高保真度的视觉模拟类似，Gazebo 提供高保真度的物理模拟，其提供一整套传感器模型，以及对用户和程序非常友好的交互方式。Gazebo 较典型的应用为设计机器人、测试机器人算法、用显示场景进行回归测试等。

3.2 机器人的 gazebo 仿真

本次实验使用 gazebo 对机器人进行仿真，通过配置软件相关参数，载入机器人模型，运行控制节点，可以在 gazebo 中观察机器人的动作情况，同时可通过指令查看相关机器人运行相关信息。

四、实验方法与步骤

4.1 gazebo 的安装与基本操作

（1）安装 gazebo 及相关依赖包

打开终端，运行如下指令：

http://gazebosim.org/tutorials?tut=install_ubuntu&cat=install

```
$ sudo apt-get install gazebo7 ros-kinetic-qt-build ros-kinetic-gazebo-ros-control
ros-kinetic-gazebo-ros-pkgs ros-kinetic-ros-control ros-kinetic-control-toolbox
ros-kinetic-realtime-tools ros-kinetic-ros-controllers ros-kinetic-xacro python-wstool
ros-kinetic-tf-conversions ros-kinetic-kdl-parser
```

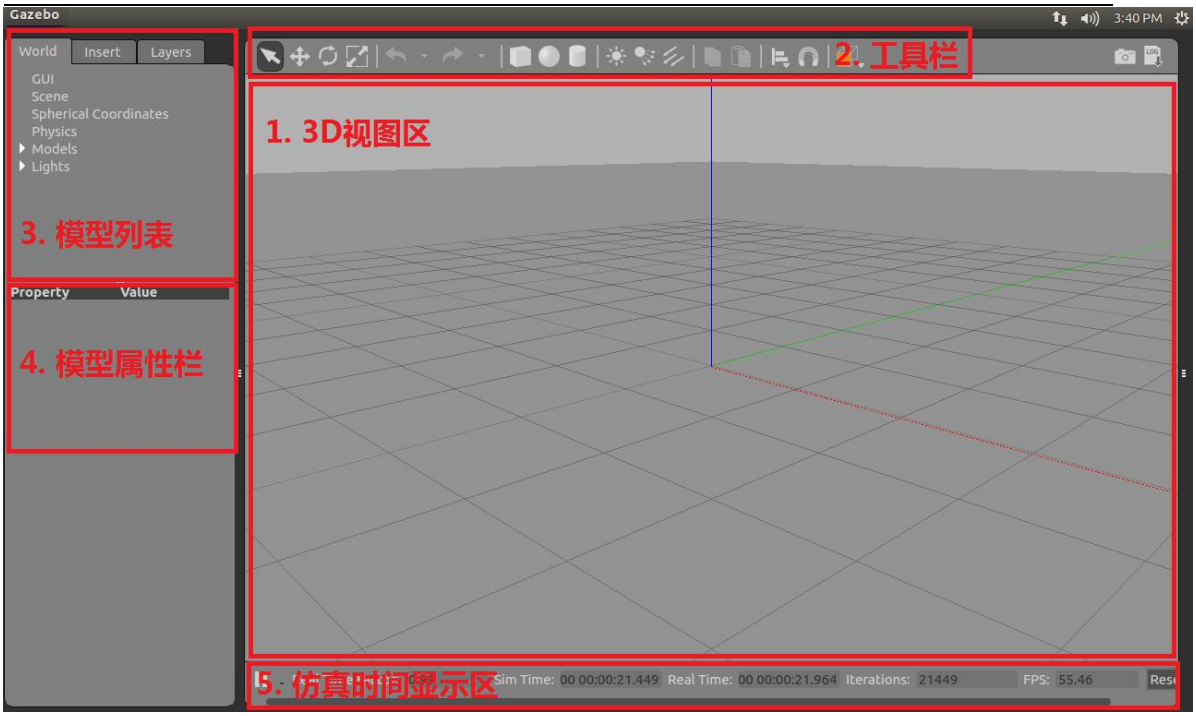
（若 ros 安装的是桌面完整版则已安装 gazebo 7.0，实验电脑若已安装完成则不需要进行再次安装）

（2）启动 gazebo

打开终端，运行指令：

```
$ roscore
$ rosrun gazebo_ros gazebo
```

启动 gazebo 后界面如下：



主界面包括以下几个部分：

1 ： 3D 视图区

2 ： 工具栏

3 ： 模型列表

4 ： 模型属性项

5 ： 时间显示区

（3）查看 gazebo 相关话题

在终端中执行如下指令：

$ rostopic list

若安装成功则显示如下话题：

/gazebo/link_states

/gazebo/model_states

/gazebo/parameter_descriptions

/gazebo/parameter_updates

/gazebo/set_link_state

/gazebo/set_model_state

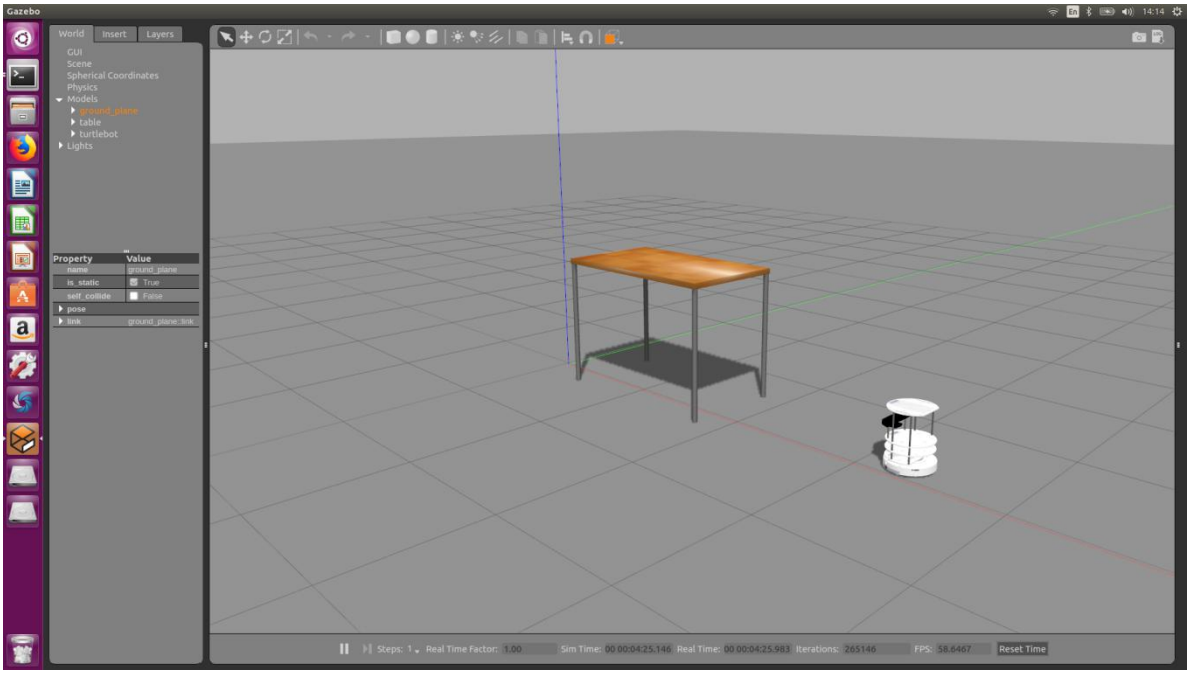
同样也可以通过查看服务列表中是否有关于 gazebo 的服务来判定是否安装成功。

$ rosservice list

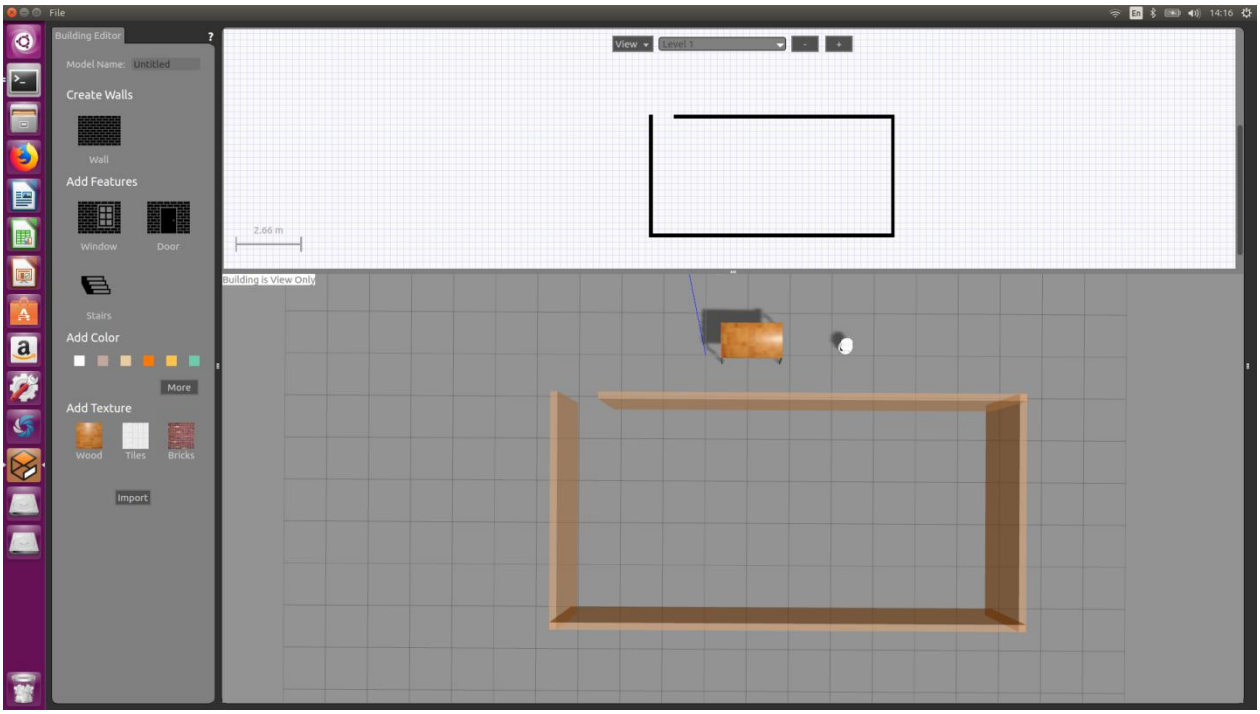
（4）gazebo 仿真环境构建

Gazebo 有两种构建仿真环境的方法，第一种方法是直接插入已有模型文件，第二种是使用工具 building editor 进行环境绘制。

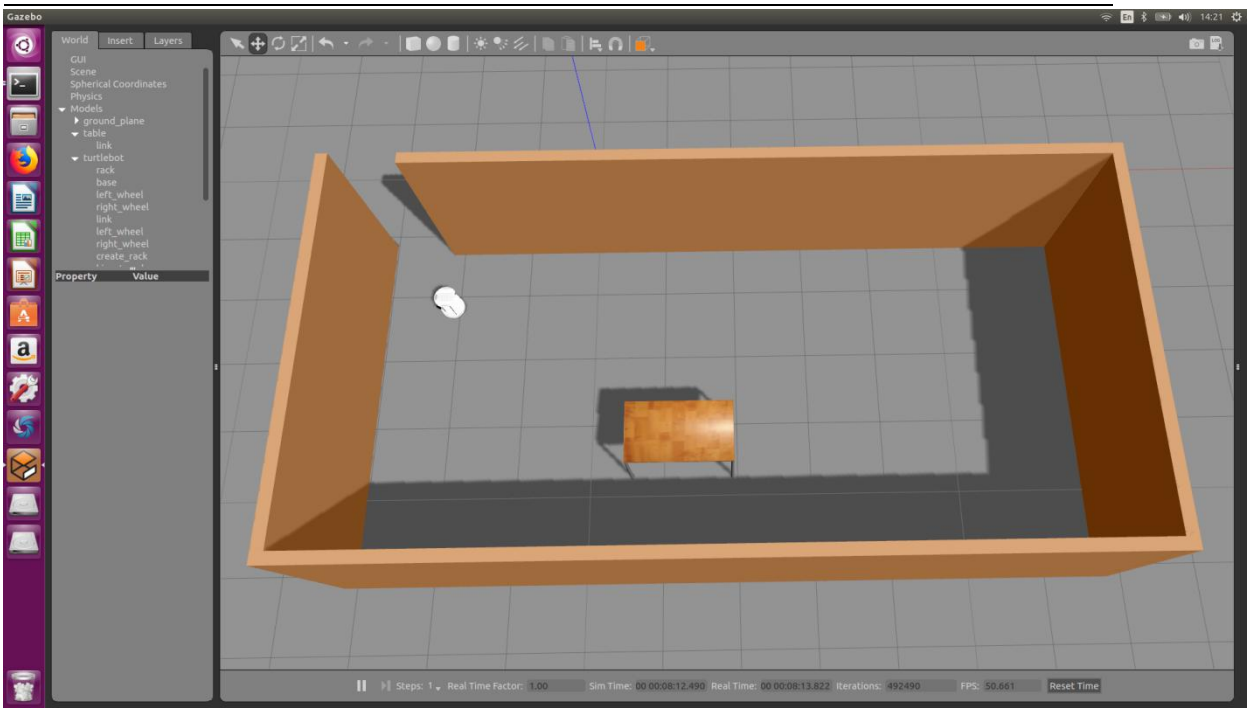
通过下载模型文件，然后解压模型到目录：~/.gazebo/models 下，即可在 gazebo 中 insert 标签插入模型；如图所示，可在环境中添加“table”、“turtlebot”：



在 gazebo 菜单 edit->building editor 打开 building editor 界面，可以根据需要画出墙体等模型，然后保存文件为模型，可用于仿真环境。如下图所示建立 5mx10m（开口为 1m）的墙体。



可以使用工具栏中的移动工具，移动“table”、“turtlebot”位置到建立的墙体围成的空间中，如下图：



4.2 机器人仿真实验

（1）机器人仿真包安装

关闭所有终端，打开新终端，安装模型文件到工作空间：

```
$ cd ~/ros_ws/src
$ wstool init .
$ wstool merge
https://raw.githubusercontent.com/RethinkRobotics/baxter_simulator/kinetic-devel/baxter_simulator.rosinstall
$ wstool update
```

编译工作空间：

```
$ source /opt/ros/kinetic/setup.bash
$ cd ~/ros_ws
$ catkin_make
$ cp src/baxter/baxter.sh .
```

（2）启动机器人仿真环境

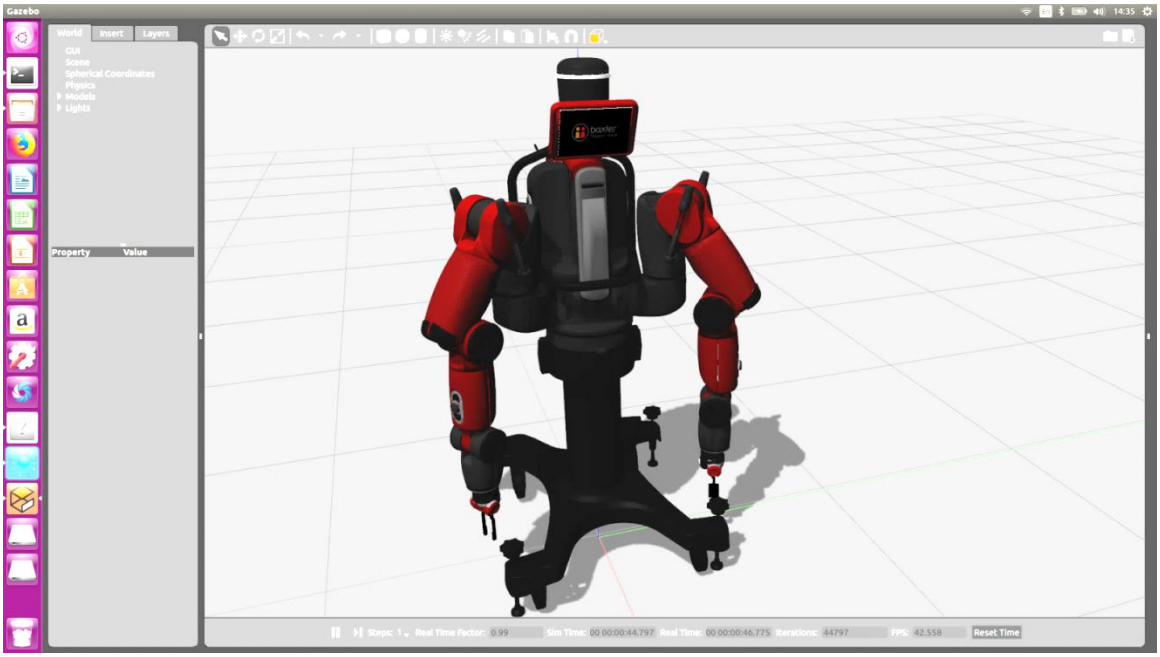
打开终端，运行环境配置脚本:

```
$ source /opt/ros/kinetic/setup.bash
$ cd ~/ros_ws
$    ./baxter.sh    sim
```

通过 launch 文件，启动 gazebo 并载入机器人仿真模型：

```
$ roslaunch baxter_gazebo baxter_world.launch
```

指令成功运行后会出现如下画面，可以看到 baxter 已载入 gazebo 的 3D 视图区，在左侧 world 标签 models 分支下可以查看到 baxter。



（3）运行机器人控制例程

打开新终端，执行以下各条指令：

```
$ source /opt/ros/kinetic/setup.bash
$ cd ~/ros_ws
$ ./baxter.sh sim
$ rosrun baxter_examples joint_position_keyboard.py
```

键盘控制机械臂运动控制的关节映射如下：

```
Esc: Quit                          ?: Help
/: left: gripper calibrate         b: right: gripper calibrate
,: left: gripper close             c: right: gripper close
m: left: gripper open              x: right: gripper open
y: left_e0 decrease                q: right_e0 decrease
o: left_e0 increase                r: right_e0 increase
u: left_e1 decrease                w: right_e1 decrease
i: left_e1 increase                e: right_e1 increase
6: left_s0 decrease                1: right_s0 decrease
9: left_s0 increase                4: right_s0 increase
7: left_s1 decrease                2: right_s1 decrease
8: left_s1 increase                3: right_s1 increase
h: left_w0 decrease                a: right_w0 decrease
l: left_w0 increase                f: right_w0 increase
j: left_w1 decrease                s: right_w1 decrease
k: left_w1 increase                d: right_w1 increase
n: left_w2 decrease                z: right_w2 decrease
.: left_w2 increase                v: right_w2 increase
```

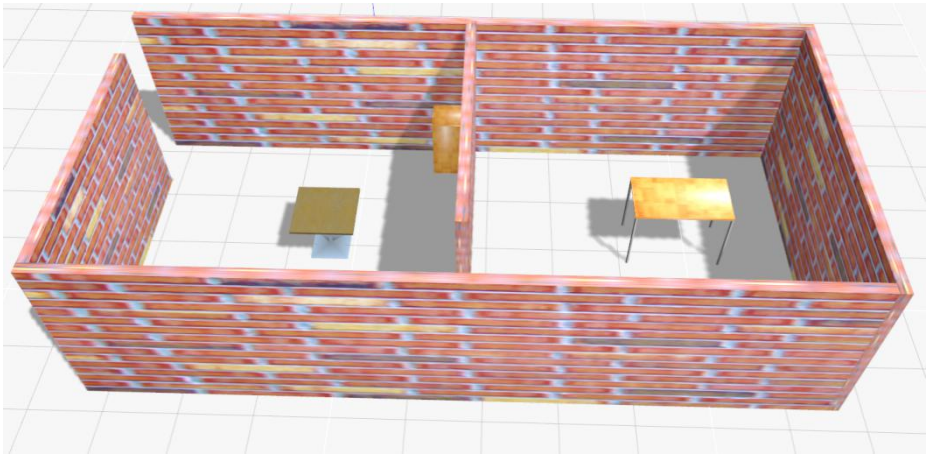
按下键盘数字“7”、“8”时，观察可以看到 gazebo 中 baxter 左臂关节随控制在运动，手臂在上下摆动；可依次按下键盘其他按键观察手臂运动。

五、实验报告

实验报告按规定的格式撰写，内容包括：

（1）实验目的

（2）简述使用 gazebo 构建下图所示的仿真环境步骤（环境组成：5mx10m 房间隔成两间，cabinet，cafe_table,table）



（3）简述键盘控制 baxter 手臂仿真实验的步骤。

六、注意事项

本实验课为公共科目，由于设备数量有限，请同学们听从实验老师的指导。注意课堂纪律、遵守实验室规章制度和设备使用规范。爱护爱惜实验设备。

实验5　机器人运动规划实验

一、实验目的

1. 了解 MoveIt 的安装与基本使用
2. 使用 MoveIt 对机器人进行运动规划与绕障

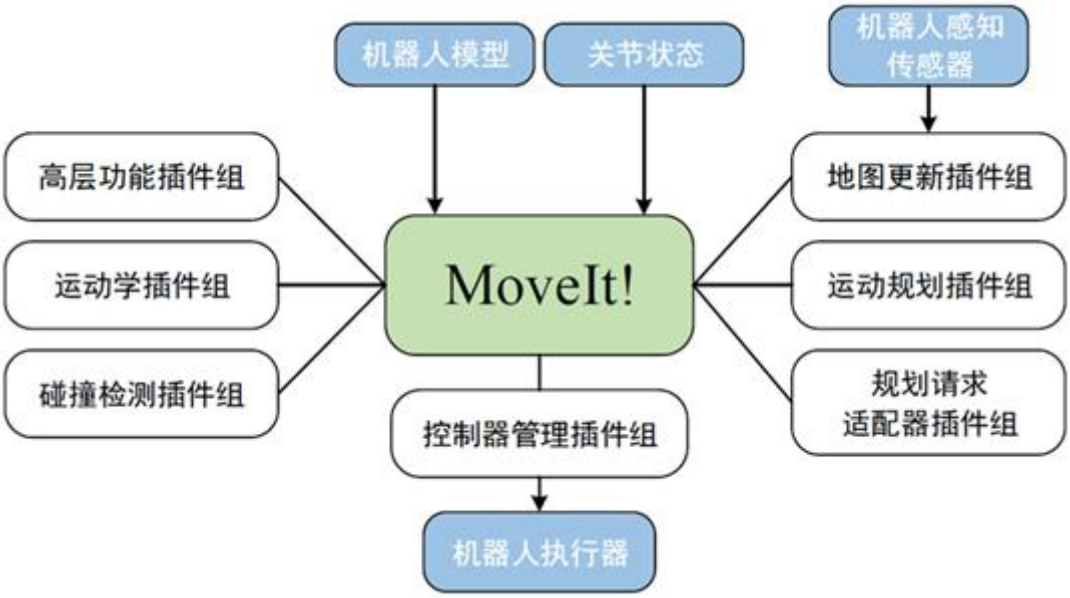
二、实验所用单元

实验电脑一台（安装 Ubuntu16.04 系统、ROS kinetic 系统,可连接互联网）、baxter 一台

三、实验内容

3.1 MoveIt 简介

Moveit 是一个易于使用的集成化开发平台，它由一系列移动操作的功能包组成（运动规划、操作控制、3D 感知、运动学、控制与导航算法等），提供友好 GUI，可应用于工业、商业、研发和其他领域。Moveit 功能包组成如下图所示：



运动学插件组是用来进行机器人运动学求解的，运动学求解器有 KDL、TRAC-IK、IKFAST，MoveIt 默认求解器是 KDL，KDL 求的是数值解，优点是可求解封闭情况下逆运动学，缺点是求解速度慢，可能找不到解。很多时候在做运动规划的时候，MoveIt 经常会提示规划失败、求解失败等错误，很多都是因为 KDL 这款运动学插件导致的；遇到这种情况，可以更换求解器。TRAC-IK 和 KDL 类似，也是一种基于数值解的运动学插件，但是在算法层面上进行了很多改进，求解效率高了很多，TRAC-IK 也有问题，每次求解得到的关节位置不一定相同。IKFAST 是一种基于解析算法的运动学插件，可以保证每次求解的一致性，求解比较稳定，速度快。IKFAST 安装过程比 KDL、TRAC-IK 要复杂，但求解效果较好。

3.2 机器人的 MoveIt 运动规划与绕障

在机器人控制系统中，MoveIt 解决机器人从当前状态运动到目标状态的运动规化问题。开发者可通过程序接口或者指令完成指令发布，本次实验考虑到可操作性采用 GUI 进行机

器人的运动规划实验。通过 GUI 指定机器人的目标位置，执行 planning 操作在机器人模型上可看到规划的路径，并可在机器人 baxter 执行， baxter 实际运行路径应与机器人模型规划的路径一致。

四、实验方法与步骤

4.1 MoveIt 的安装与基本操作

打开终端，运行如下指令对 moveit 进行安装：

```
$ sudo apt-get update
$ sudo apt-get install ros-kinetic-moveit
```

在终端运行如下指令，安装机器人的 moveit 配置软件包：

```
$cd ~/ros_ws/src
$ git clone https://github.com/ros-planning/moveit_robots.git
$ cd ~/ros_ws/
$ catkin_make
```

编译完成后，可开启 MoveIt 进行 moveit 基本操作；

（1）开启 baxter 机器人，确保机器人与实验电脑网络通畅，可进行正常联机操作；

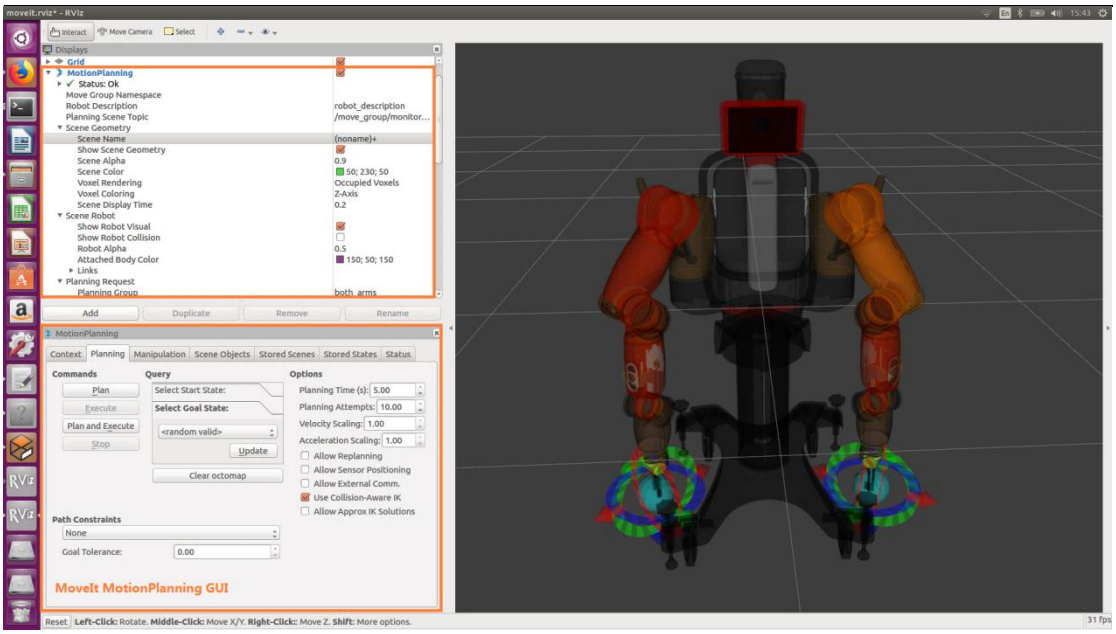
（2）开启终端，运行如下指令，进行机器人使能操作，运行关节轨迹动作服务器；

```
$ cd ~/ros_ws
$ ./baxter.sh
$ rosrun baxter_tools enable_robot.py –e
$ rosrun baxter_interface joint_trajectory_action_server.py
```

（3）开启终端，运行如下指令，启动 moveit 例程

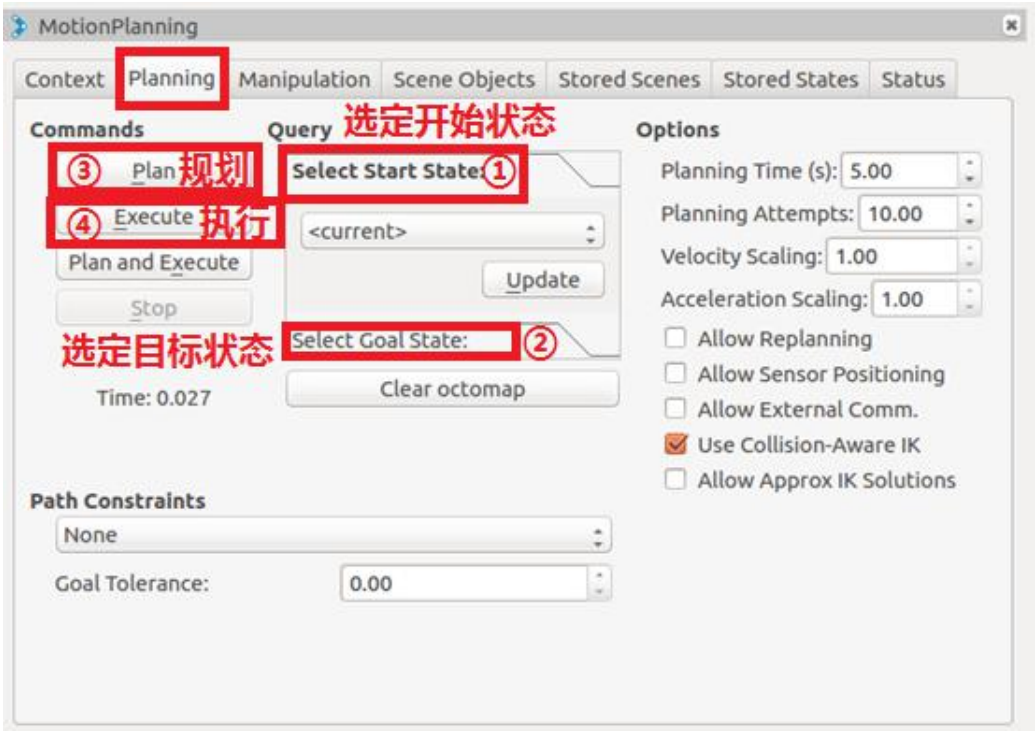
```
$ cd ~/ros_ws
$ ./baxter.sh
$ roslaunch baxter_moveit_config baxter_grippers.launch
```

指令运行结束后会出现下图所示窗口：



从窗口中可以看出，baxter 机器人已载入 rviz 模型显示区，同时两手臂末端有姿态调整工具，左侧显示区可以看到 moveit 插件 GUI 界面，实验中主要使用 MotionPlanning GUI 的 Planning、Scene Objects 标签。Planning 主要用于指定运动起始状态，设定运动目标状态，执行运动规划并进行运动执行。

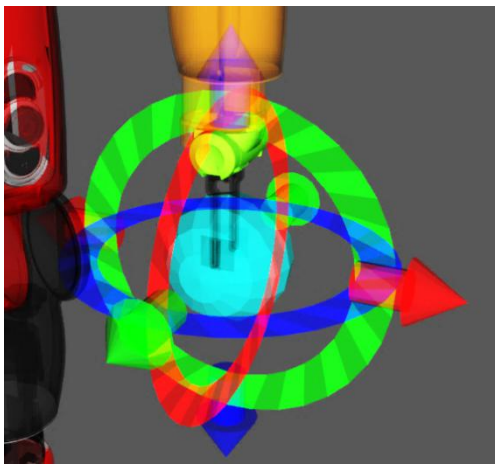
下图为 MotionPlanning Planning 标签界面



标签①“Select Start State”用于选定机器人的开始状态，运动规划时使用机器人实际所在位置作为开始状态，若机器人实际位置不在指定的开始状态需将位置调整到指定的开始状态；下拉菜单可选项：random valid、random、current、same as goal；选定 current；

标签②“Select goal State”用于选定机器人目标状态，若有已命名保存的状态点则选择该状态点即可，没有则保持默认选项；可通过鼠标拖动机器人末端状态调整工具实现目标状态的设置。末端工具包含 7 个控件：调整手臂末端空间位置的球；调整手臂姿态的红、绿、

蓝三个圆环及调整特定方向位移的红、绿、蓝三个箭头，通过鼠标移动到相应控件拖动即可对机器人手臂进行状态调整。



设定机器人目标状态后，使用按钮③“Plan”启动机器人运动规划，若规划成功则会显示 time：xxx；若运动规划失败则显示 Failed，当规划失败后，可以再次点击 Plan 按钮进行规划；运动规划成功后 rviz 3d 显示区可以看到机器人模型的运行轨迹；

运动规划成功后，点击④“Execute”按键，此时可以看到与实验电脑相连的机器人 baxter 已经开始运动了，运动轨迹与机器人模型的运行轨迹一致，目标状态完全一致。

MoveIt 求解器切换

找到 baxter_moveit_config 功能包，进入 config 文件夹，打开 kinematics.yaml 文件，修改“kinematics_solver”值为：kdl_kinematics_plugin/KDLKinematicsPlugin，则选择 KDL 解算器进行求解；下图所示为 kinematics.yaml 文件内容：

kinematics.yaml (~/catkin_ws/src/moveit_robots/baxter/baxter_moveit_config/config) - gedit

Open Save

kinematics.yaml × ompl_planning.yaml ×

```
1 left_arm:
2   kinematics_solver: kdl_kinematics_plugin/KDLKinematicsPlugin
3 # uncomment to use ikfast kinematics plugin
4 # kinematics_solver: baxter_left_arm_kinematics/IKFastKinematicsPlugin
5   kinematics_solver_attempts: 3
6   kinematics_solver_search_resolution: 0.005
7   kinematics_solver_timeout: 0.005
8 right_arm:
9   kinematics_solver: kdl_kinematics_plugin/KDLKinematicsPlugin
10 # uncomment to use ikfast kinematics plugin
11 # kinematics_solver: baxter_right_arm_kinematics/IKFastKinematicsPlugin
12   kinematics_solver_attempts: 3
13   kinematics_solver_search_resolution: 0.005
14   kinematics_solver_timeout: 0.005
```

同样，打开 kinematics.yaml 文件，修改“kinematics_solver”值为：trac_ik_kinematics_plugin/TRAC_IKKinematicsPlugin，则切换为 TRAC_IK 进行求解。下图所示为 kinematics.yaml 文件内容：

kinematics.yaml (~/catkin_ws/src/moveit_robots/baxter/baxter_moveit_config/config) - gedit

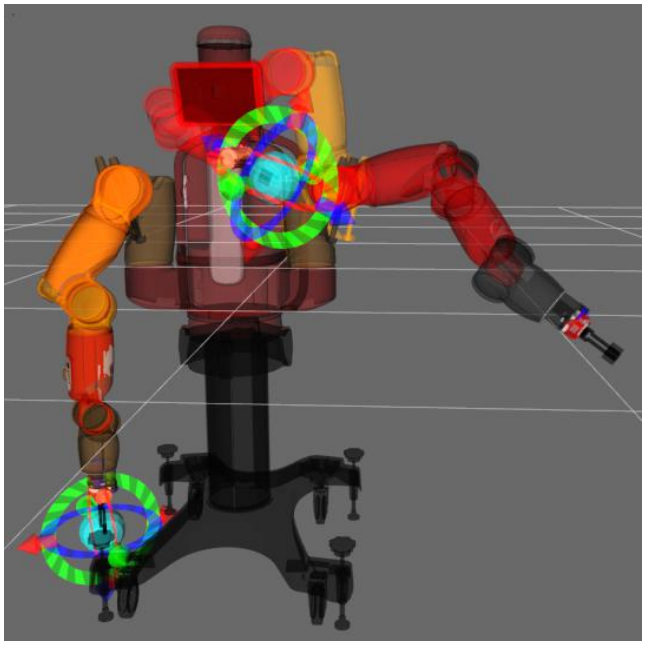
Open Save

```
left_arm:
#  kinematics_solver: kdl_kinematics_plugin/KDLKinematicsPlugin
# uncomment to use ikfast kinematics plugin
# kinematics_solver: baxter_left_arm_kinematics/IKFastKinematicsPlugin
# uncomment to use TRAC-IK kinematics plugin
  kinematics_solver: trac_ik_kinematics_plugin/TRAC_IKKinematicsPlugin
  kinematics_solver_attempts: 3
  kinematics_solver_search_resolution: 0.005
  kinematics_solver_timeout: 0.005
right_arm:
#  kinematics_solver: kdl_kinematics_plugin/KDLKinematicsPlugin
# uncomment to use ikfast kinematics plugin
# kinematics_solver: baxter_right_arm_kinematics/IKFastKinematicsPlugin
# uncomment to use TRAC-IK kinematics plugin
  kinematics_solver: trac_ik_kinematics_plugin/TRAC_IKKinematicsPlugin
  kinematics_solver_attempts: 3
  kinematics_solver_search_resolution: 0.005
  kinematics_solver_timeout: 0.005
```

修改求解器配置文件后，重新启动 MoveIt 即可进行机器人 baxter 手臂的运动规划。

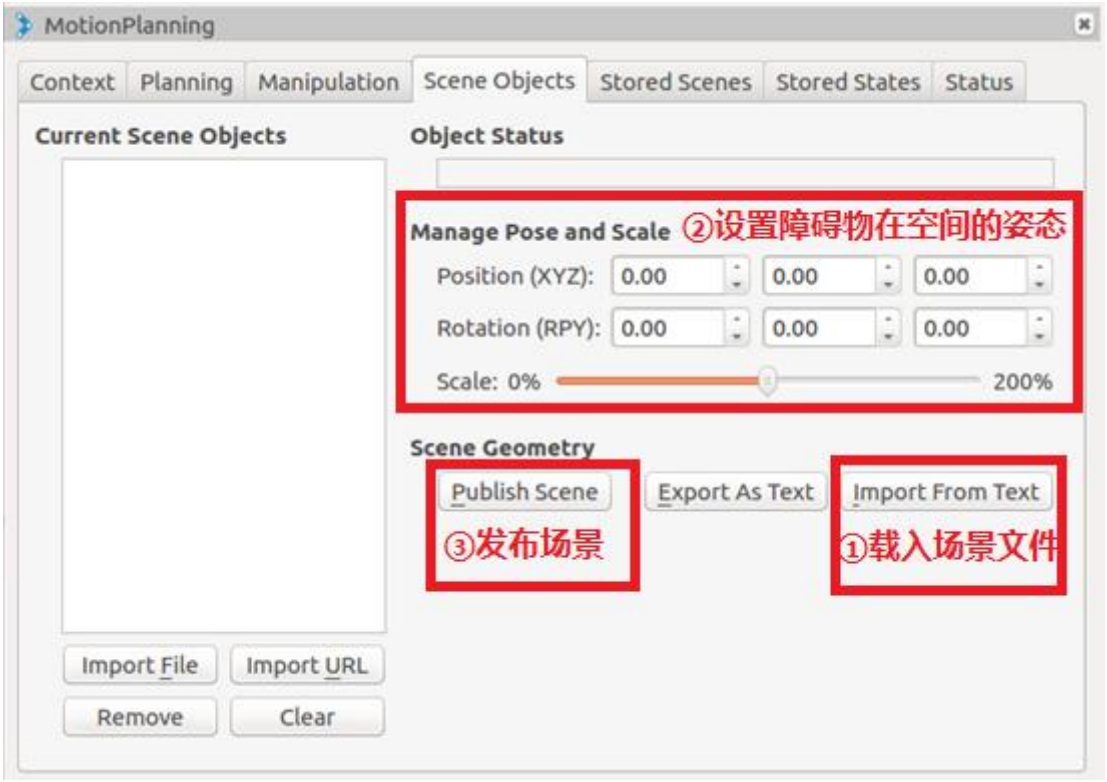
Moveit 的碰撞检测

在使用 MoveIt 做运动规划的过程中，使用鼠标调整设置机器人目标状态时，会出现下图所示状况，拖动的手臂由橙色变成了红色，红色区域表示机器人手臂模型与其他部分发生干涉，也是表示运动过程中会产生碰撞，在这种情况下若进行 Plan 操作，是无法成功的进行运动规划的，遇到这种情况时需要调整机器人状态避免干涉后，重新进行 Plan 操作。



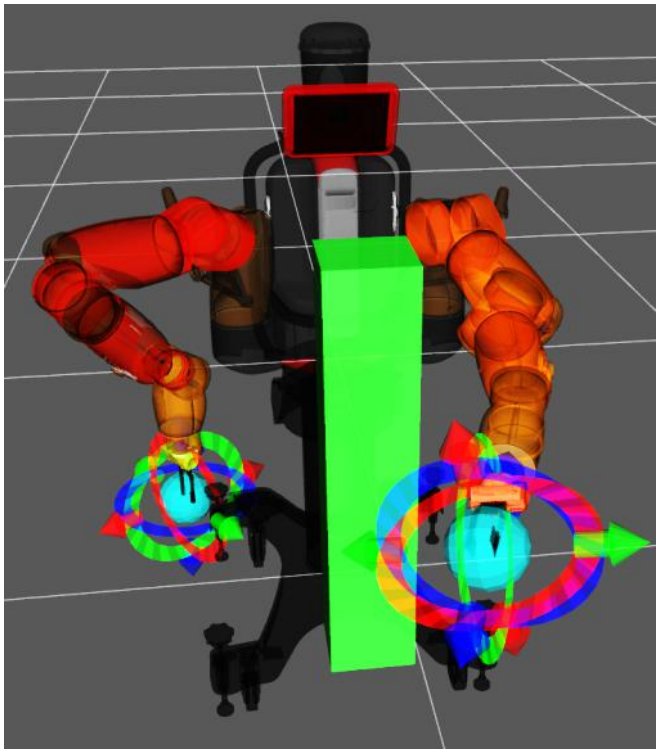
MoveIt 场景加入障碍物进行路径规划

在 MoveIt 场景中加入障碍物需要使用 MotionPlanning GUI 的 Scene Objects 标签，界面如下：



（1）载入障碍物场景文件。通过点击 Import From Text 按钮按照 baxter_moveit_config 包路径找到场景文件 baxter_pillar.scene，包文件路径为：~/catkin_ws/src/moveit_robots_kinetic-devel/baxter/baxter_moveit_config/baxter_scenes/baxter_pillar.scene

(2)通过“Manage Pose and Scale”选框内的参数调整障碍物 pillar 与 baxter 的相对位置，调整到 baxter 胸前的位置，如下图所示，为了便于观察 baxter 的碰撞检测的运动规划展示，将机器人左手臂放置于障碍物左侧。



（3）障碍物位置设定完成后，需要发布场景，直接点击界面中的“Publish Scene”按钮发布场景。

（4）把机器人手臂运行到当前位置，使 baxter 机器人的实际位置与模型中的机器人状态一致。

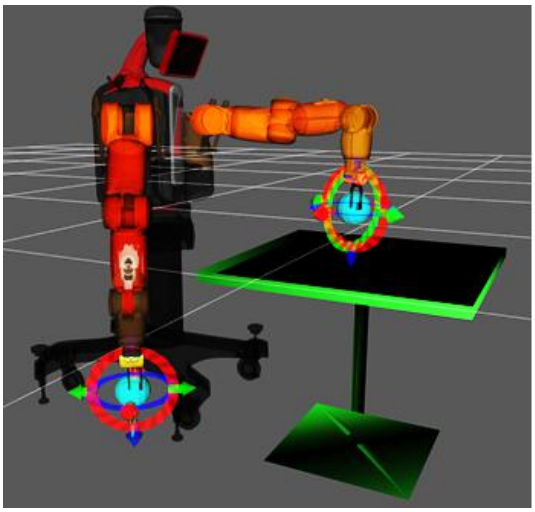
（5）以机器人左手臂在障碍物的左侧为起点，然后调整机器人左手臂的状态到障碍物的右侧，注意机器人手臂状态不能与障碍物发生干涉，以机器人左手位于障碍物右侧的状态作为手臂运动的目标状态，使用“Plan”按钮进行运动规划，若运动规划成功则可直接使用“Execute”按钮直接执行，执行的过程中可以观察机器人运行的状态是否具有绕过障碍物。

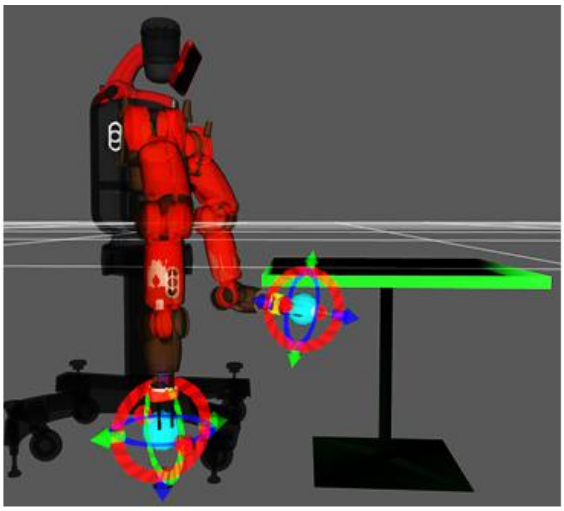
（6）作为对比，可以在“Scene Objects”标签所在界面左侧“Current scene objects”框内选定障碍物，然后使用下方“remove”按钮移除障碍物，然后发布场景，将手臂再次拖动到之前障碍物左侧的位置，进行运动规划，并执行观察运动状态是否与之前有障碍物时的一样。

五 实验报告

1.实验目的

2.参照实验内容，建立下图所示场景，分别将手臂末端置于桌子上方、下方进行 MoveIt 运动规划，观察 baxter 机器人手臂运动状态。





使用 MotionPlanning GUI 的 Scene Objects 标签“import file”按钮载入“Cafe_table.dae”模型，路径为：/home/cothink/.gazebo/models/cafe_table/meshes/cafe_talbe.dae 文件加载完成后需要使用“Publish Scene”按钮发布场景然后进行运动规划操作；

3 简述 MoveIt 进行运动规划基本步骤；

六 注意事项

本实验课为公共科目，由于设备数量有限，请同学们听从实验老师的指导。注意课堂纪律、遵守实验室规章制度和设备使用规范。爱护爱惜实验设备。

实验6　MoveIt 环境感知实验

一、实验目的

1. 了解深度相机基本使用
2. 使用 MoveIt 进行环境感知、运动规划与绕障

二、实验所用单元

实验电脑一台（安装 Ubuntu16.04 系统、ROS kinetic 系统,可连接互联网）、baxter 一台、kinect 一套；

三、实验内容

3.1 MoveIt 环境感知简介

Moveit 通过调用组件的方式可以对环境进行感知，使用深度相机感知环境数据，生成一种被称为 octomap 的方格地图，感知后的环境可以作为环境障碍物，机械臂运动规划时会主动避开有 octomap 地图的地方。其中使用的 moveit 组件 Occupancy Map Updater，Occupancy Map Updater 接受两种数据：（1）pointcloud2 点云；（2）深度图 sensor_msgs/Image。

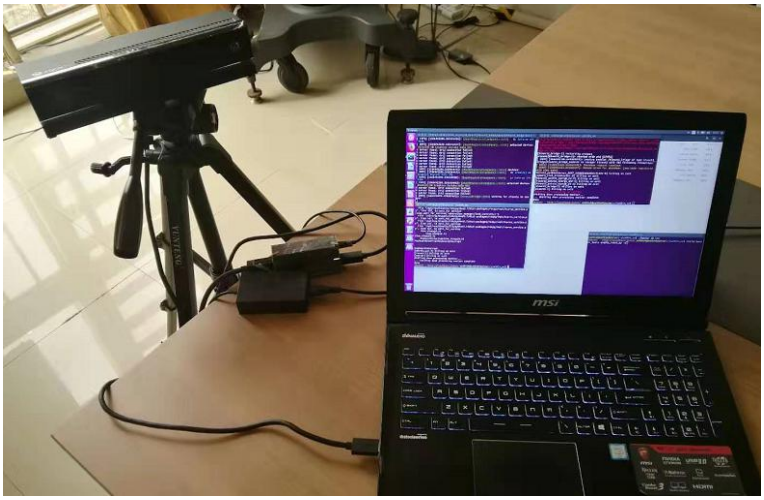
3.2 基于 MoveIt 环境感知功能的运动规划与绕障

本实验主要使用 kinect 数据来展示 moveit 的 3d 感知功能，3D 感知的方式采用 pointcloud2 点云，同时结合实际场景，使用 baxter 进行运动规划，可查看运动规划避障的过程。

四 实验方法与步骤

4.1 MoveIt 环境感知功能

（1）连接 kinect 电源线，连接 kinect 与实验电脑 USB 数据线；



（2）确保实验电脑已安装 kinect 驱动及 ros 驱动，打开终端，开启 kinect 相应 ros 节点。在终端输入如下指令：

$ roslaunch　kinect2_bridge　kinect2_bridge.launch

运行后界面如下：

```
/home/cothink/catkin_ws/src/iai_kinect2/kinect2_bridge/launch/kinect2_bridge.launch
[ INFO] [1566549747.216702227]: [DepthRegistrationOpenCL::init]   1: GeForce GTX
 960M
[ INFO] [1566549747.216731199]: [DepthRegistrationOpenCL::init] selected device:
 Intel(R) HD Graphics Skylake Halo GT2
X server found. dri2 connection failed!
X server found. dri2 connection failed!
X server found. dri2 connection failed!
X server found. dri2 connection failed!
X server found. dri2 connection failed!
X server found. dri2 connection failed!
X server found. dri2 connection failed!
[ INFO] [1566549747.391408969]: [DepthRegistrationOpenCL::init] devices:
[ INFO] [1566549747.391435758]: [DepthRegistrationOpenCL::init]   0: Intel(R) HD
 Graphics Skylake Halo GT2
[ INFO] [1566549747.391575349]: [DepthRegistrationOpenCL::init]   1: GeForce GTX
 960M
[ INFO] [1566549747.391608567]: [DepthRegistrationOpenCL::init] selected device:
 Intel(R) HD Graphics Skylake Halo GT2
X server found. dri2 connection failed!
X server found. dri2 connection failed!
X server found. dri2 connection failed!
[ INFO] [1566549747.483281355]: [Kinect2Bridge::main] waiting for clients to con
nect
```

（3）打开终端，查看相应的 ros 话题是否有相应数据输出；输入如下指令，查看 pointcloud2 点云话题：

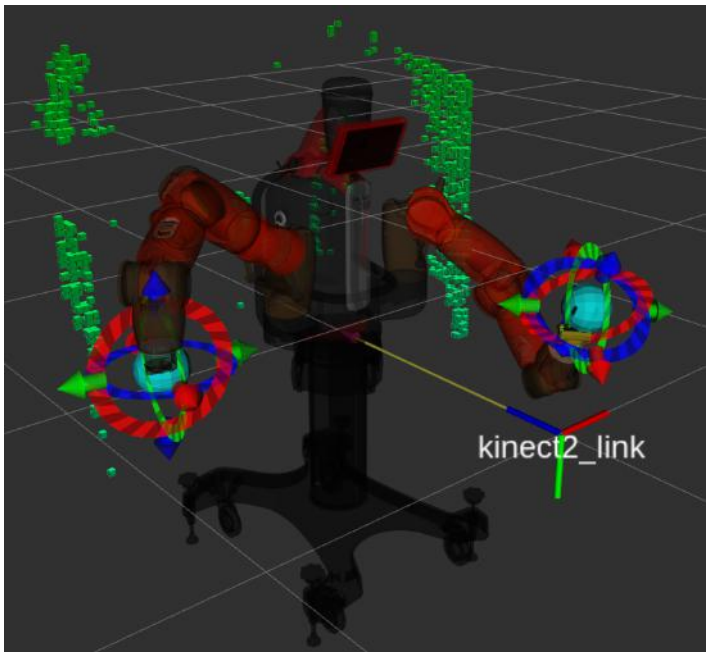
$rostopic list | grep points

```
cothink@cothinkServer: ~
cothink@cothinkServer:~$ rostopic list | grep points
/kinect2/hd/points
/kinect2/qhd/points
/kinect2/sd/points
cothink@cothinkServer:~$
```

（4）开启 moveit，查看 octomap 方格地图；连接 baxter 与实验电脑，运行如下指令：

$roslaunch baxter_moveit_config demo_kinect.launch

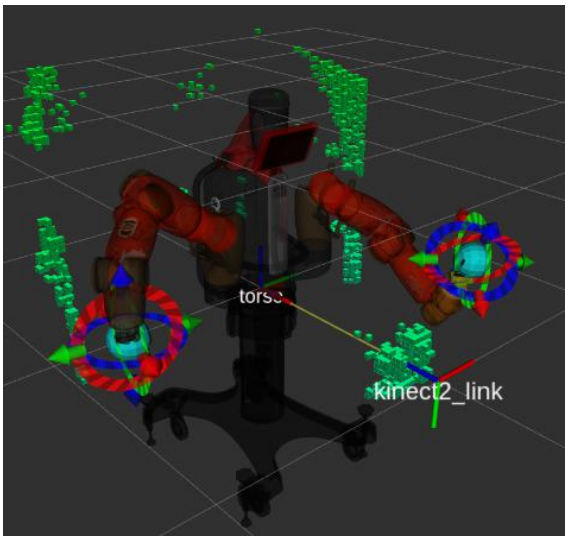
运行后可看到机器人后侧的障碍物情况。



（5）在 kinect 视野中放置障碍物，观察 octomap 方格地图的动态变化；下图中可以看到 kinect 相机前放置的纸盒实时的 rviz 模型中显示出来，通过实时 octomap 地图可实现 kinect 视野中

的环境感知。





4.2 基于 MoveIt 环境感知功能的运动规划与绕障

（1）连接实验电脑与 baxter 机器人，连接 kinect 相机；

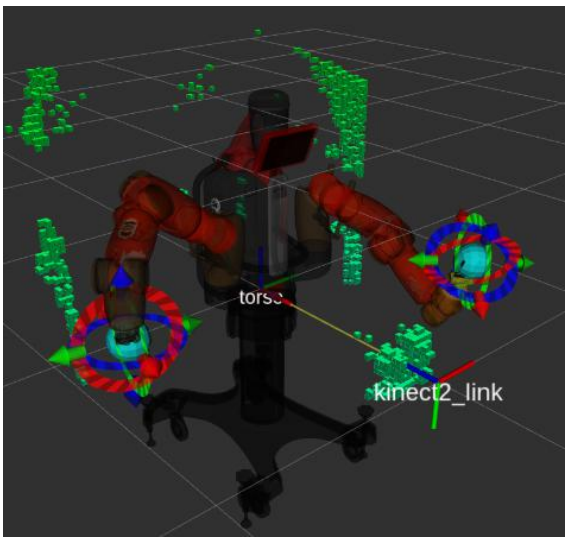
（2）布置 kinect 与机器人的方位：kinect 相机面向 baxter，kinect 相机位于 baxter 正前方 1m 处，高度与 baxter 腰部安装面平齐；

（3）开启 moveit 启动 launch 文件，观察 rviz 模型显示区内环境是否与实际场景一致，可通过调整 kinect 相机的位置使得看到的点云数据与 baxter 机器人位置基本重合即可。

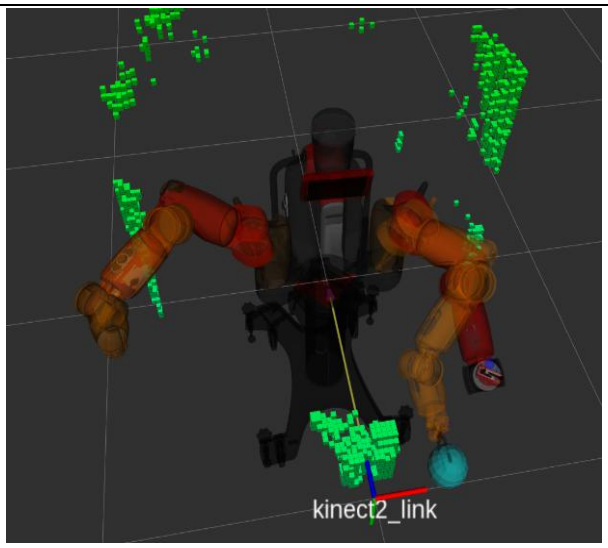
$ roslaunch baxter_moveit_config demo_baxter.launch kinect:=true

（4）在 kinect 视野中放置障碍物，在 moveit 中设置运动规划的起始位置，进行路径规划，路径规划成功后观察 baxter 手臂实际运行状态，是否有绕开障碍物的动作。参考下图所示环境，在 kinect 前方设置一纸盒（其他障碍物也行），octomap 地图中识别后可开始 moveit 运动规划。

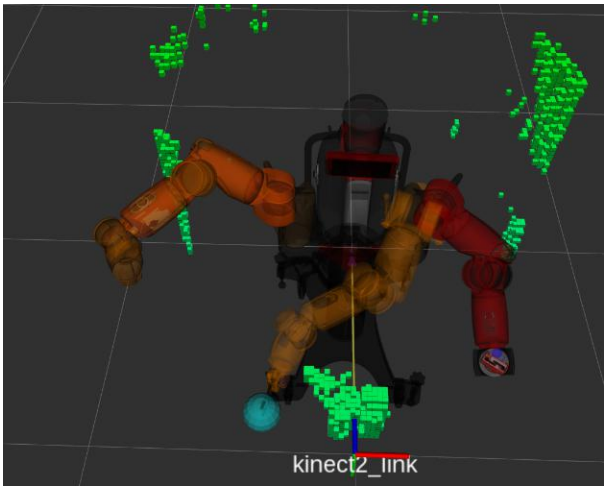




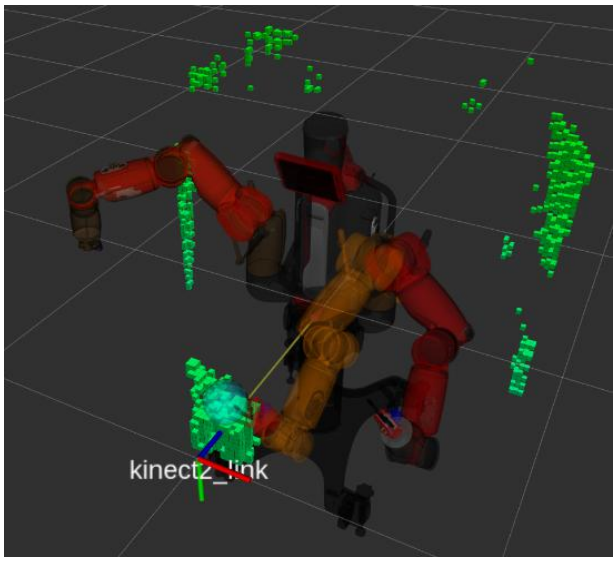
以 baxter 当前位置为运动规划初始状态，将手臂运到规划到障碍物左侧位置，以此位置作为作为实验的运动规划起始状态。



拖动 baxter 机器人手臂末端，跨过障碍物将末端移动到障碍物右侧，设置此状态为运动规划的目标状态，保持 kinect 视野中障碍物不要被走，进行路径规划，观察机械臂运动规划避障情况。



在设置运动规划目标状态拖动 baxter 左臂过程中，可以观察到当 baxter 末端被拖动到与纸盒干涉时，手臂末端变红，表明手臂进行 moveit 运动规划时碰撞检测会出错，运动规划将失败，在进行运动规划时应避免干涉的情况发生。



（5）更换其他障碍物，进行从障碍物上方到下方的运动规划实验；观察机械臂运动规划绕障情况。

五、实验报告

1.实验目的

2.参照实验内容，使用 kinect 深度相机实现 moveit 环境感知功能，简述基本步骤，保存场景图片及 rviz 中模型场景图片，查看 octomap 地图与实景放置障碍物情况一致性。

六、注意事项

本实验课为公共科目，由于设备数量有限，请同学们听从实验老师的指导。注意课堂纪律、遵守实验室规章制度和设备使用规范。爱护爱惜实验设备。

实验 7　RVIZ 使用

一、实验目的

1.了解 rviz 软件功能用途

2.学习机器人传感器数据可视化应用

二、实验所用单元

Baxter 机器人一台；

电脑一台（内存 4G 以上、至少 20G 可用硬盘空间）

路由器一台

网线 2 条。

三、实验内容

1.了解 rviz 软件功能用途

2.机器人传感器数据可视化添加操作

四、实验方法和步骤

1.Rviz 三维可视化平台

机器人运行时会产生大量数据，但是数据形态的值不利于开发者去感受数据所描述的内容，所以需要将数据可视化显示。ROS 为用户提供了一款显示多种数据的三维可视化平台—rviz。

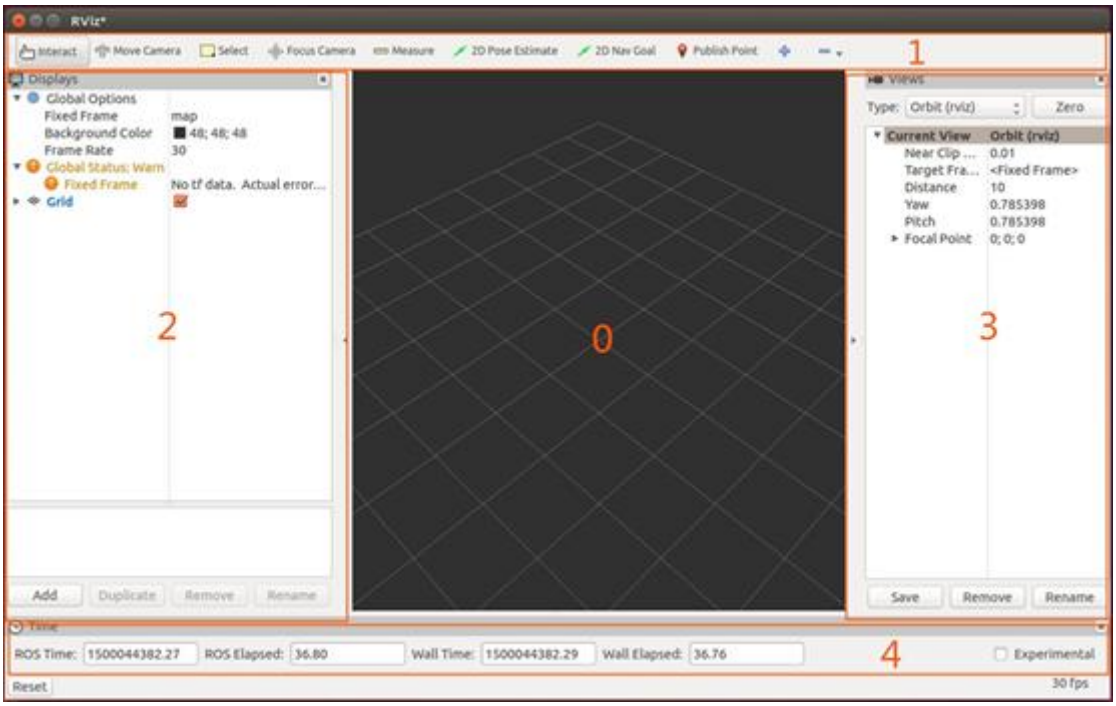
rviz 很好的兼容了各种基于 ROS 软件框架的机器人平台。在 rviz 中，可以对机器人、周围物体等任何实物进行尺寸、质量、位置、关节等属性的描述，并且在界面中呈现出来。同时，rviz 还可以通过图形化的方式，实时显示机器人传感器的信息、机器人的运动状态、周围环境的变化等。

rviz 使用

rviz 已经集成在桌面完整版的 ROS 系统当中，如果已经成功安装了桌面完整版的 ROS，在终端中分别运行如下命令即可启动 ROS 系统和 rviz 平台：

```
$ roscore
$ rosrun rviz
```

启动成功的 rviz 主界面如下图所示：



该界面主要包含以下几个部分：

0 ： 3D 视图区，用于可视化显示数据，目前没有任何数据，所以显示黑色

1 ： 工具栏，提供视角控制、目标设置、发布地点等工具

2 ： 显示项列表，用于显示当前选择的显示插件，可以配置每个插件的属性

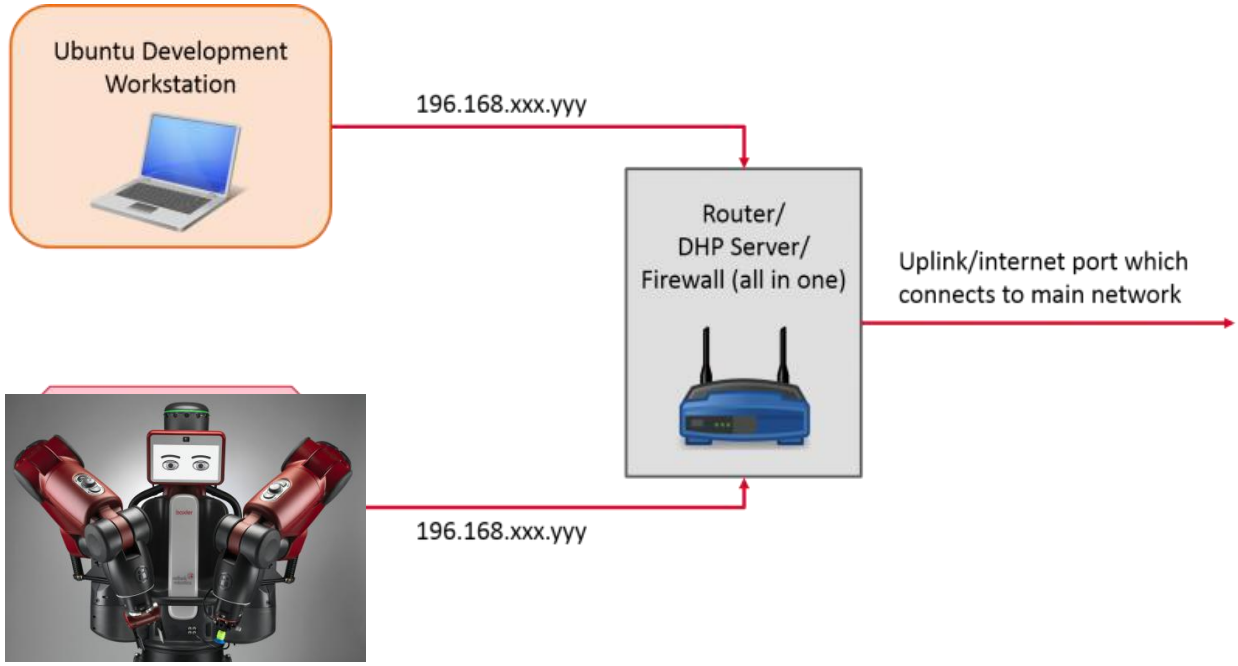
3 ： 视角设置区，可以选择多种观测视角

4 ： 时间显示区，显示当前的系统时间和 ROS 时间

进行数据可视化的前提当然是要有数据，我们通过 Baxter 丰富的传感器来感受 Rviz 强大的数据可视化能力。

2.Baxter 机器人添加可视化数据

如下所示，将 Baxter 机器人和电脑通过网线链接的方式链接到路由器局域网上。



1）、打开新的终端，执行 SDK 运行环境命令：

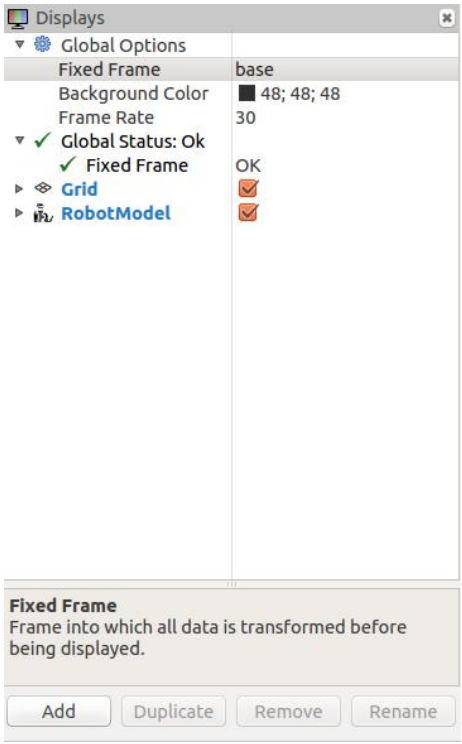
$ cd ~/ros_ws

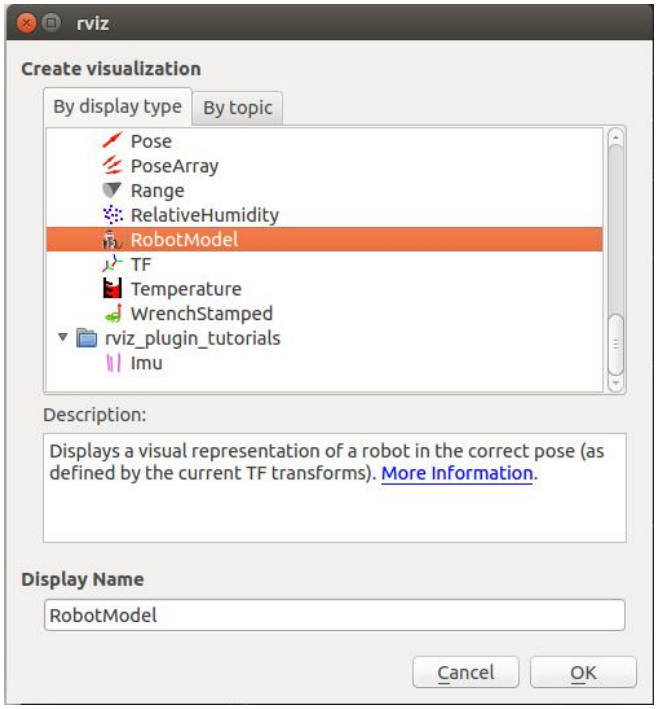
$ ./baxter.sh

2）、启用 rviz：

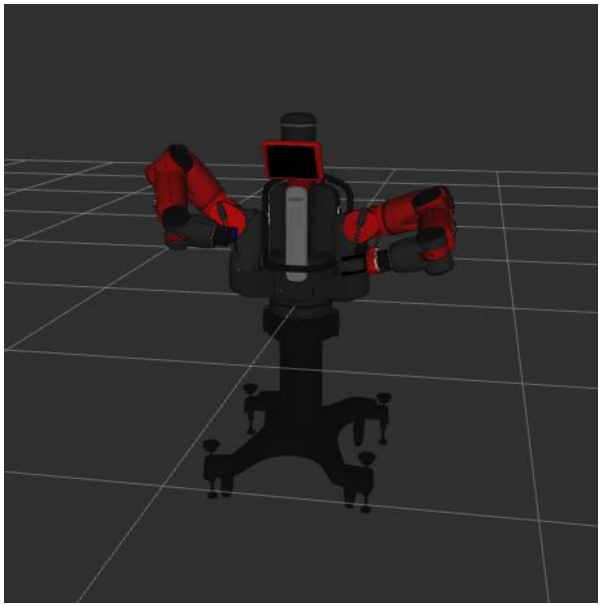
$ rosrun rviz rviz

首先，点击 rviz 界面左侧下方的“Add”按键，选择添加 RobotModel，然后在 Fixed Frame 中将参考坐标选择为 base，以机器人的的基座原点为视角。如下图所示：

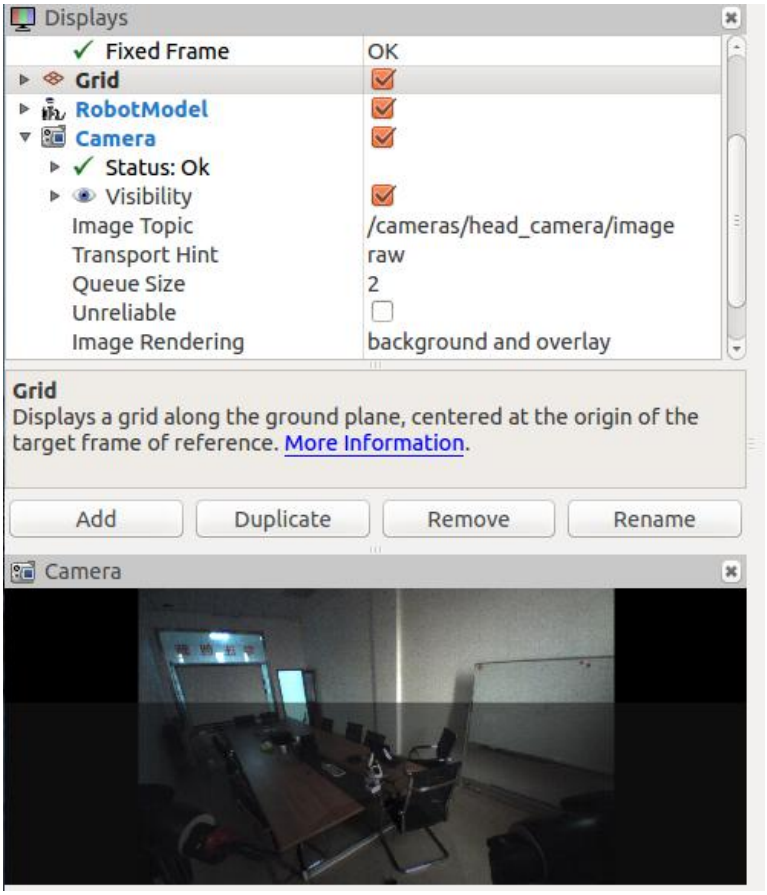




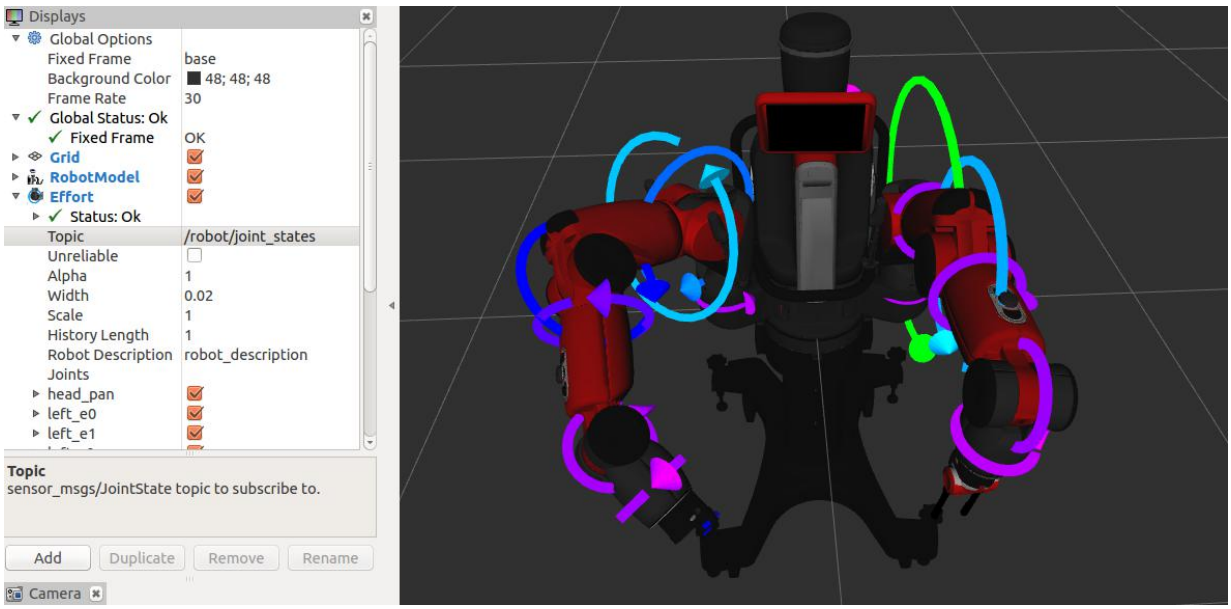
添加后 Baxter 模型将显示在 rviz 中，并且保持与实体 Baxter 机器人姿态实时一致。



然后我们添加 camera 组件，添加后在 Image Topic 中选择机器人的摄像头话题 /camera/head_camera/image，选择添加后机器人头部摄像头的图像就会显示在一个独立窗口中，如下图：

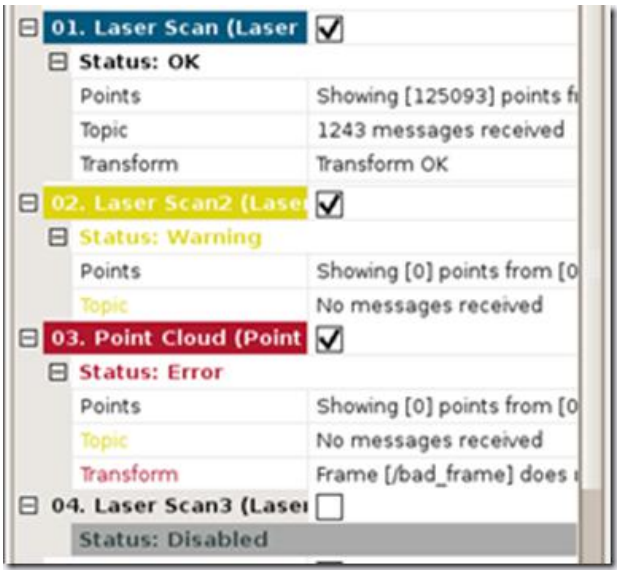


Effort 组件是用于可视化机器人力矩数据，添加 Effort 组件后在 Topic 中选择 /robot/joint_states，机器人手臂每个关节的力矩数据就将显示如下：



力矩数据同样是实时显示的，我们可以使用零重力键移动手臂或在手臂保持姿态时轻微推拉各个关节，观察各关节力矩大小的变化。

如果以上显示有问题，请检查属性区域的“Status”状态。Status 有四种状态：OK、Warning、Error 和 Disabled，如果显示的状态不是 OK，那么请查看错误信息，并详细检查数据发布是否正常。



五、实验报告

本实验要求完成以下内容：

4. 了解 rviz 软件的用途
5. 学习添加可视化组件
6. 完成 rviz 中添加 Baxter 的手臂红外测距、头部声呐探测，屏幕显示等功能。

六、注意事项

本实验课为公共科目，由于设备数量有限，请同学们听从实验老师的指导。注意课堂纪律、遵守实验室规章制度和设备使用规范。爱护爱惜实验设备。

实验8 dobot 写字实验

一、实验目的

1.了解 dobot 机器人的 ros 环境配置
2.了解 ros 中 dobot 机器人功能包的开发

二、实验所用单元

实验电脑一台（安装 Ubuntu16.04 系统、ROS kinetic 系统,可连接互联网）、dobot、写字套件。

三、实验内容

3.1 了解 dobot 运动模式

Dobot 机械臂运动模式如下几种运动功能：JOG 功能、PTP 功能、CP 功能、ARC 功能。JOG 功能指定 dobot 机械臂朝指定方向或关节运行，适合调整姿态；PTP 可实现点到点运动，并且具有 MOVJ，MOVL 和 JUMP 等方式，可用于书写，尤其是 JUMP 模式，可以直接从上一笔结束点跳到下一笔起点，自动添加提笔动作；CP 功能可实现点到点运动，单没有 JUMP 方式，需要组合多个动作才能完成 JUMP 功能；ARC 功能主要用于实现圆弧运动，应用于写字不合适。

通过了解各运动功能，比较而言，采用 PTP 功能用于书写应用更合适，本实验采用 PTP 功能完成机械臂书写控制。

3.2 运用 dobot 接口进行写字操作

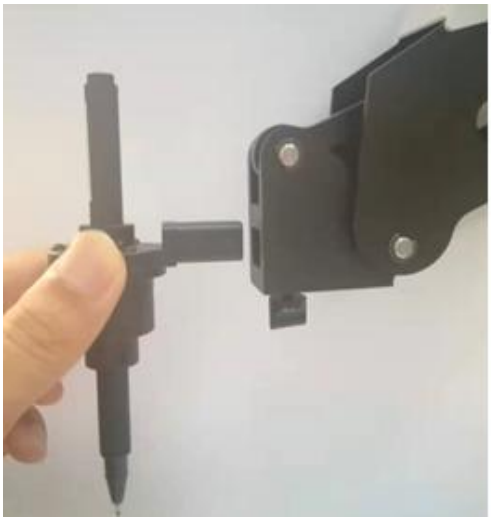
文字笔画的书写通过控制机械臂末端运动完成，使用 PTP 功能 MOVL 模式；笔画之间的跳转使用 PTP 功能 JUMP 模式完成。

本实验主要通过 domot 机械臂不同运动模式控制完成文字书写的任务。实验中主要用 PTP 功能 MOVL 模式、JUMP 模式及实时位姿获取功能。

四、实验方法与步骤

4.1 dobot 写字套件安装

将笔安装到夹笔器中，固定牢靠，然后将写字套件安装到 dobot 机械臂末端，拧紧锁紧螺丝，确保写字过程中写字套件不会脱出。




4.2 dobot ros 功能包安装与配置

（1）Dobot 机械臂 ros 功能包下载

将实验电脑开机进入 window 系统，使用如下下载链接即可下载 dobot 官方 ros 功能包。

http://download.dobot.cc/development-protocol/dobot-magician/win7-win10/cn/Dobot-Magician-ROS-Demo.zip

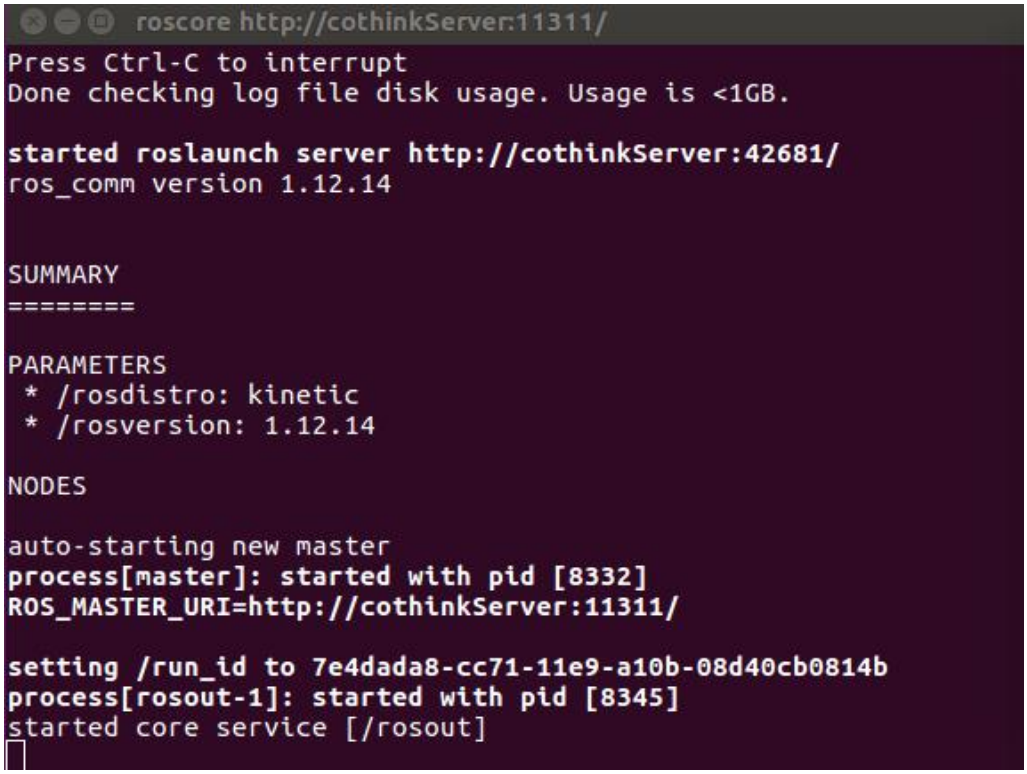
下载完成后需将压缩包拷贝进入 Ubuntu 16.04 后进行解压使用。

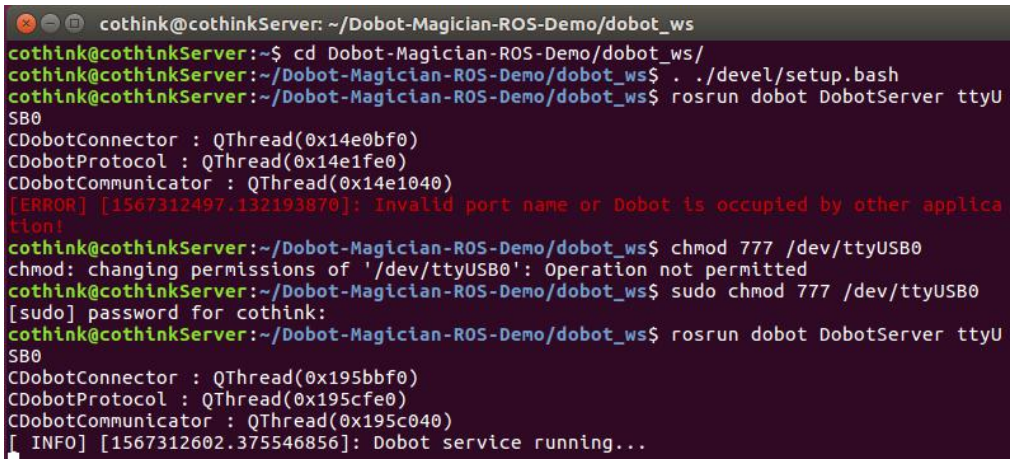
（2）ros 包编译与运行

在 Linux 系统下将 dobot_ws 解压到~目录下，解压后打开终端，进入 dobot_ws 目录，使用 catkin_make 指令编译工作空间：

```
cothink@cothinkServer: ~/Dobot-Magician-ROS-Demo/dobot_ws
[ 98%] Generating C++ code from dobot/SetTRIGCmd.srv
[ 98%] Built target dobot_generate_messages_cpp
Scanning dependencies of target dobot_generate_messages
[ 98%] Built target dobot_generate_messages
Scanning dependencies of target dobot_gencpp
[ 98%] Built target dobot_gencpp
Scanning dependencies of target DobotClient_JOG
[ 98%] Building CXX object dobot/CMakeFiles/DobotClient_JOG.dir/src/DobotClient_JOG.cpp.
o
[ 98%] Linking CXX executable /home/cothink/Dobot-Magician-ROS-Demo/dobot_ws/devel/lib/d
obot/DobotClient_JOG
[ 98%] Built target DobotClient_JOG
Scanning dependencies of target DobotClient_PTP
[ 98%] Building CXX object dobot/CMakeFiles/DobotClient_PTP.dir/src/DobotClient_PTP.cpp.
o
[100%] Linking CXX executable /home/cothink/Dobot-Magician-ROS-Demo/dobot_ws/devel/lib/d
obot/DobotClient_PTP
[100%] Built target DobotClient_PTP
Scanning dependencies of target DobotServer
[100%] Building CXX object dobot/CMakeFiles/DobotServer.dir/src/DobotServer.cpp.o
[100%] Linking CXX executable /home/cothink/Dobot-Magician-ROS-Demo/dobot_ws/devel/lib/d
obot/DobotServer
[100%] Built target DobotServer
cothink@cothinkServer:~/Dobot-Magician-ROS-Demo/dobot_ws$
```

连接 USB 插头与实验电脑，打开终端执行 source 命令，然后运行 ros 节点程序。如下图所示：



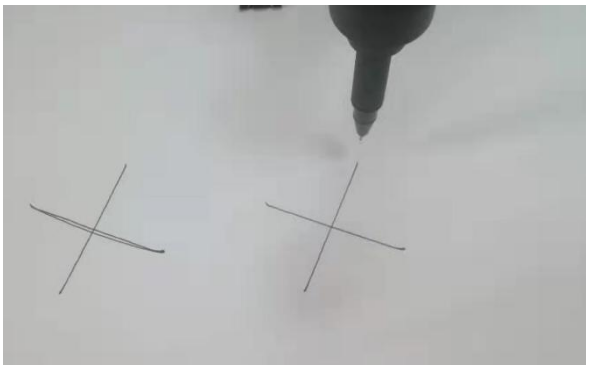


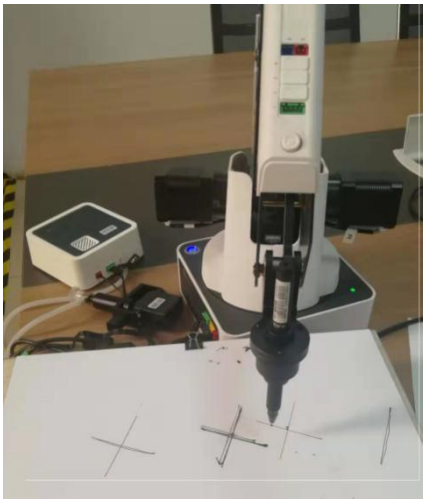


此时可观察到 dobot 机械臂已开始进行运动。

4.3 dobot 写“十”字

dobot 写“十”字功能包为 DobotClient_Write,此功能包是基于 DobotClient_PTP 功能包修改，该功能包的基本使用如下：dobot 上电，与实验电脑相连，按住 unlock 键拖动 dobot 机械臂末端笔尖到书写板上，然后运行 DobotClient_Write 节点，可观察到机械臂在书写板上书写“十”字，当拖动 dobot 机械臂到新位置时，机械臂会在新的位置书写“十”字。书写效果如下图所示：





（1）dobot 接口的使用

Dobot 机械臂 ros 功能包可以通过调用服务的方式完成各机械臂各功能的调用。下面是本次实验中使用的接口情况：

- 设置 JUMP 模式提笔高度，此处设置为 20。

```
// Set PTP jump parameters
do {
    client = n.serviceClient<dobot::SetPTPJumpParams>("/DobotServer/SetPTPJumpParams");
    dobot::SetPTPJumpParams srv;

    srv.request.jumpHeight = 20;
    srv.request.zLimit = 200;
    client.call(srv);
} while (0);
```

- 获取机械臂末端当前坐标，将获取的坐标分别赋值给 x，y，zVal；

```
//request Z axis value from service GetPose
client1 = n.serviceClient<dobot::GetPose>("/DobotServer/GetPose");
dobot::GetPose srv6;
client1.call(srv6);                    //get current pose

//zVal , Z axis value from service GetPose response
float zVal =srv6.response.z;
float x = srv6.response.x;
float y = srv6.response.y;
```

- 建立 PTP 功能服务客户端 client，用于运动指令下发

```
client = n.serviceClient<dobot::SetPTPCmd>("/DobotServer/SetPTPCmd");
dobot::SetPTPCmd srv;
```

- 设置 JUMP 模式，目标点坐标设置为（x，y+25）；

```
//jump position
srv.request.ptpMode = 0;        //JUMP MODE
srv.request.x = x;
srv.request.y = y+25;
srv.request.z = zVal;
srv.request.r = 0;
client.call(srv);
while(srv.response.result != 0) {;}
```

- 设置 MOVL 模式，目标点坐标设置为（x，y-25）；

```
//jump position
srv.request.ptpMode = 2;        //MOVL MODE
srv.request.x = x;
srv.request.y = y-25;
srv.request.z = zVal;
srv.request.r = 0;
client.call(srv);
while(srv.response.result != 0) {;}
```

（2）书写“十”字程序代码解析



五、实验报告

1.实验目的

2.修改实验代码使用 MOVJ 书写十字，根据书写的效果对比两种方式书写的差别

3.参照实验步骤，设计 ros 功能包，书写文字“二”。

4.简述 JOG 功能、PTP 功能、CP 功能、ARC 功能差异及应用，并尝试使用 ARC 功能画半径为 10 的圆。

六 注意事项

本实验课为公共科目，由于设备数量有限，请同学们听从实验老师的指导。注意课堂纪律、遵守实验室规章制度和设备使用规范。爱护爱惜实验设备。

实验9 机器人控制

一、实验目的

1.了解机器人 API 接口
2.学习通过程序控制机器人运动

二、实验所用单元

Baxter 机器人一台；

电脑一台（内存 4G 以上、至少 20G 可用硬盘空间）

路由器一台

网线 2 条。

三、实验内容

1.了解机器人 API 接口

2.通过 Python 程序控制机器人运动

四、实验方法和步骤

1.SDK 与 API

API 的概念

API 即“应用程序编程接口”，是一些预先定义的函数，目的是作为“介面”沟通两个不同的东西，提供应用程序与开发人员基于某软件或硬件得以访问一组例程的能力，而又无需访问源码，或理解内部工作机制的细节。

SDK 的概念

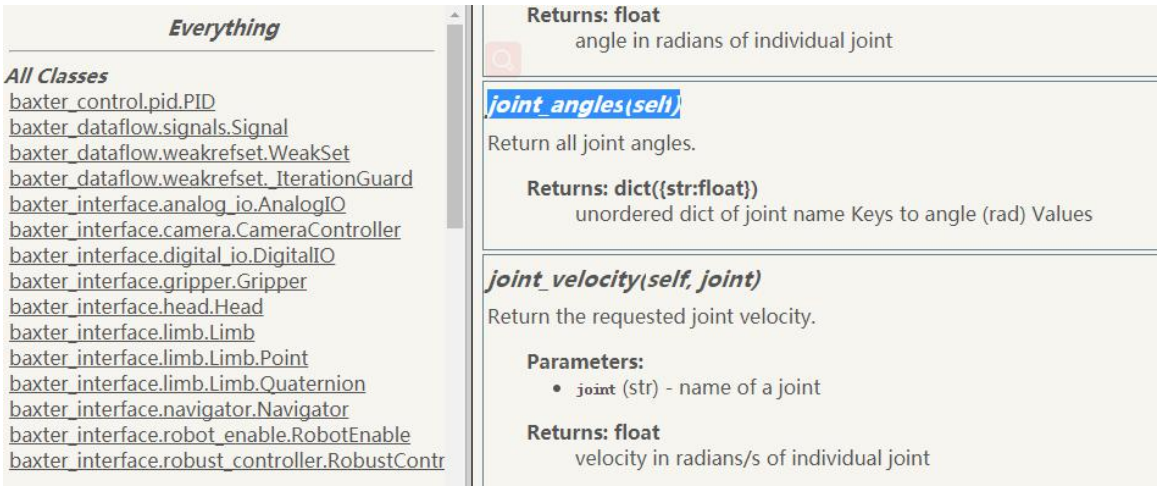
SDK 即“软体开发工具包”，一般是一些被软件工程师用于为特定的软件包、软件框架、硬件平台、操作系统等建立应用软件的开发工具的集合。通俗点是指由第三方服务商提供的实现软件产品某项功能的工具包。

SDK 和 API 区别与联系

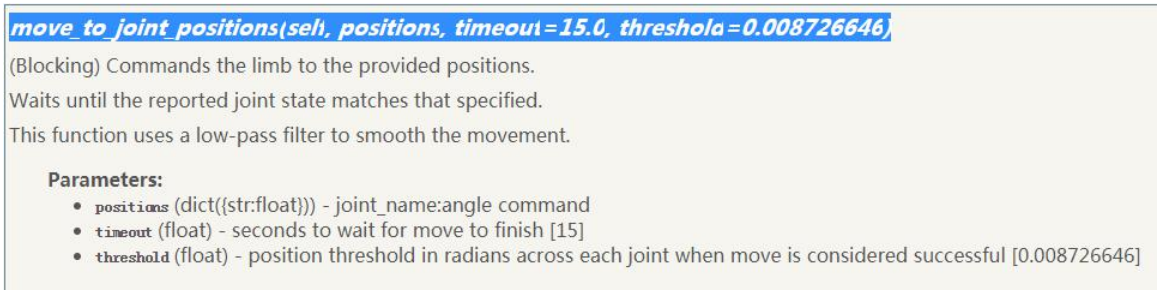
SDK 相当于开发集成工具环境，API 就是数据接口。在 SDK 环境下调用 API 数据。实际上 SDK 包含了 API 的定义，API 定义一种能力，一种接口的规范，而 SDK 可以包含这种能力、包含这种规范。但是 SDK 又不完全只包含 API 以及 API 的实现，它是一个软件工具包，它还有很多其他辅助性的功能。

Baxter 的 API 文档链接：http://api.rethinkrobotics.com/

使用 joint_angles 和 move_to_joint_positions 两个功能函数我们可以完成 Baxter 机器人打招呼的动作实现。



joint_angles 用于获取 Baxter 手臂当前各个关节的关节角度，单位为 rad。



move_to_joint_positions 可以让 Baxter 机器人手臂运行到设定的关节角度。

启动机器人仿真环境

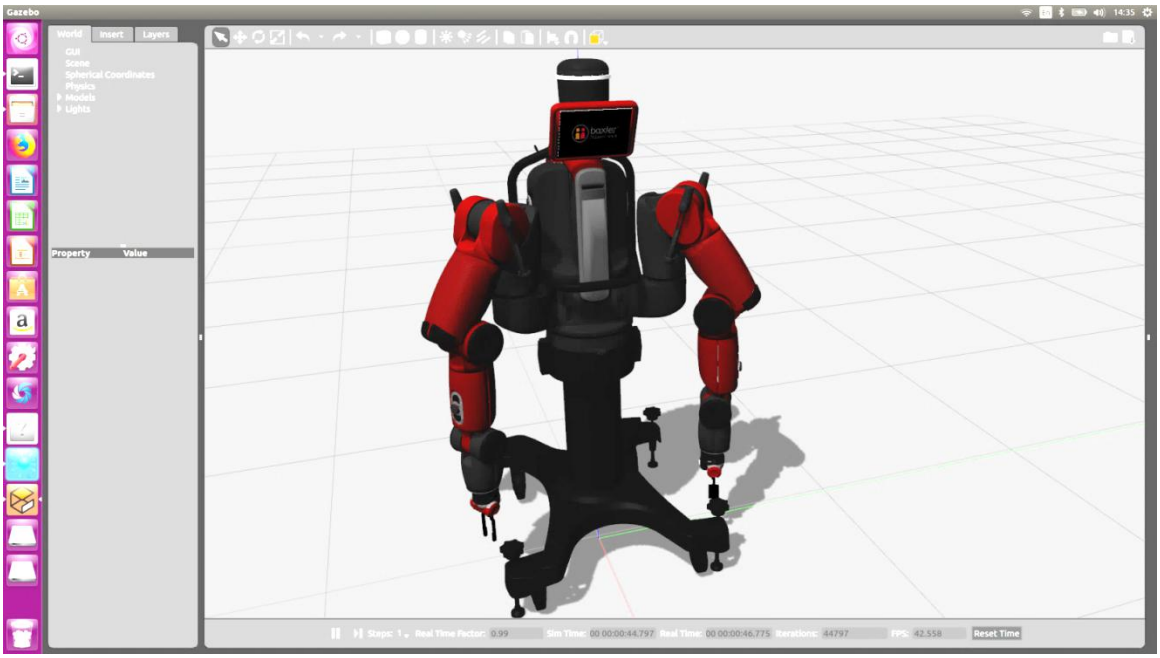
打开终端，运行环境配置脚本:

```
$ source /opt/ros/kinetic/setup.bash
$ cd ~/ros_ws
$    ./baxter.sh    sim
```

通过 launch 文件，启动 gazebo 并载入机器人仿真模型：

```
$ roslaunch baxter_gazebo baxter_world.launch
```

指令成功运行后会出现如下画面，可以看到 baxter 已载入 gazebo 的 3D 视图区，在左侧 world 标签 models 分支下可以查看到 baxter。



（3）运行机器人控制例程

打开新终端，执行以下各条指令：

```
$ cd ~/ros_ws
$    ./baxter.sh    sim
```

在 ros_ws 文件夹中新建文档，输入以下代码并获取文件程序执行权限：

```
$ python

# Import the necessary Python modules

# rospy - ROS Python API
>>> import rospy

# baxter_interface - Baxter Python API
>>> import baxter_interface

# initialize our ROS node, registering it with the Master
>>> rospy.init_node('Hello_Baxter')

# create an instance of baxter_interface's Limb class
>>> limb = baxter_interface.Limb('right')

# get the right limb's current joint angles
>>> angles = limb.joint_angles()
```

```
# print the current joint angles
>>> print angles

# reassign new joint angles (all zeros) which we will later command to the limb
>>> angles['right_s0']=0.0
>>> angles['right_s1']=0.0
>>> angles['right_e0']=0.0
>>> angles['right_e1']=0.0
>>> angles['right_w0']=0.0
>>> angles['right_w1']=0.0
>>> angles['right_w2']=0.0

# print the joint angle command
>>> print angles

# move the right arm to those joint angles
>>> limb.move_to_joint_positions(angles)

# Baxter wants to say hello, let's wave the arm

# store the first wave position
>>> wave_1 = {'right_s0': -0.459, 'right_s1': -0.202, 'right_e0': 1.807, 'right_e1': 1.714, 'right_w0':
-0.906, 'right_w1': -1.545, 'right_w2': -0.276}

# store the second wave position
>>> wave_2 = {'right_s0': -0.395, 'right_s1': -0.202, 'right_e0': 1.831, 'right_e1': 1.981, 'right_w0':
-1.979, 'right_w1': -1.100, 'right_w2': -0.448}

# wave three times
>>> for _move in range(3):
...         limb.move_to_joint_positions(wave_1)
...         limb.move_to_joint_positions(wave_2)


# quit
>>> quit()
```

运行后 Baxter 机器人将会挥动手臂向我们打招呼。

五、实验报告

本实验要求完成以下内容：

1.了解 SDK 与 API 的关系，

2.在 gazebo 中完成 Baxter 机器人 hello world 程序，

3.在 gazebo 中完成 Baxter 机器人握手动作程序。

六、注意事项

本实验课为公共科目，由于设备数量有限，请同学们听从实验老师的指导。注意课堂纪律、遵守实验室规章制度和设备使用规范。爱护爱惜实验设备。

实验10 视觉图像传递

一、实验目的

1.了解 OpenCV
2.学习图像获取函数使用
3.学习图像传递函数使用

二、实验所用单元

Baxter 机器人一台；

电脑一台（内存 4G 以上、至少 20G 可用硬盘空间）

路由器一台

网线 2 条。

三、实验内容

1.了解 OpenCV

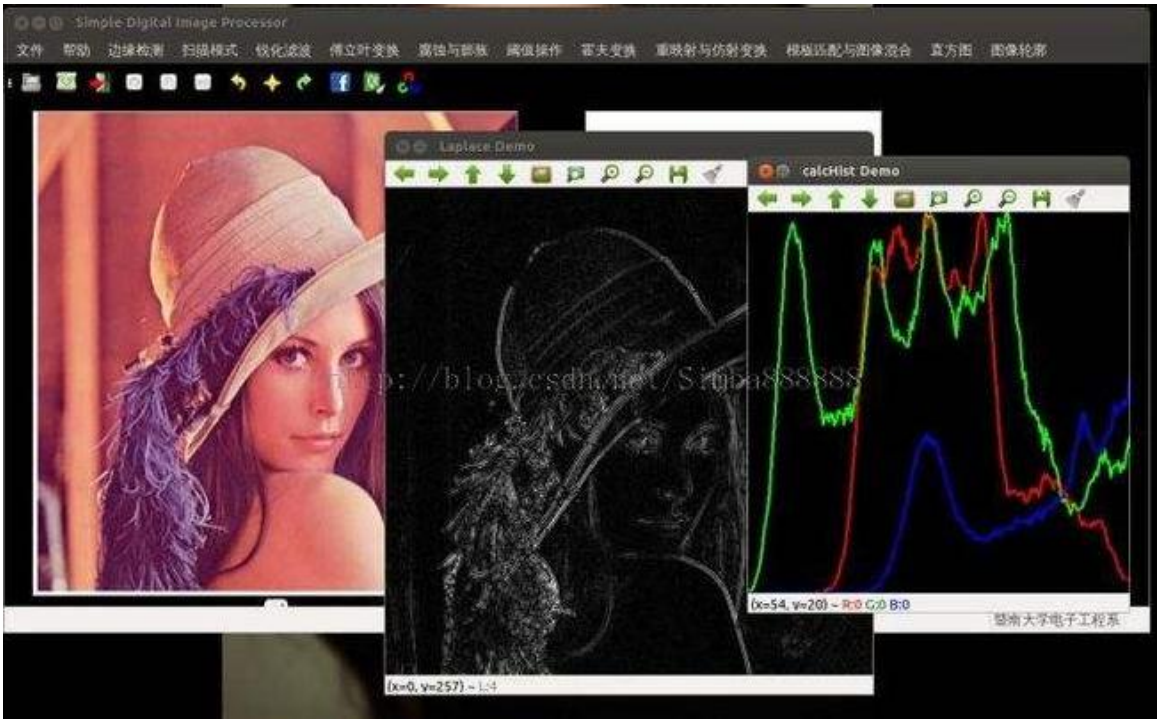
2.学习使用 OpenCV 中 IMread 函数功能

3.学习使用 OpenCV 中 CV_bridge 函数功能

四、实验方法和步骤

1.OpenCV 是什么

OpenCV 是一个用于图像处理、分析、机器视觉方面的开源函数库. 全称是 Open source Computer Vision Library,是一个开放源代码计算机视觉库。也就是说，它是一套关于计算机视觉的开放源代码的 API 函数库。 它的一个目标是提供友好的机器视觉接口函数，从而使得复杂的机器视觉产品可以加速面世。该库包含了横跨工业产品检测、医学图像处理、安防、用户界面、摄像头标定、三维成像、机器视觉等领域的超过 500 个接口函数。近年来，opencv 在入侵检测、特定目标跟踪、目标检测、人脸检测、人脸识别、人脸跟踪等领域可谓大显身手，而这些，仅仅是其应用的冰山一角。



OpenCV 经典应用—边缘检测　　OpenCV 的官方链接：https://opencv.org/

OpenCV imread 函数

imread 函数作用

imread 为 image read 的缩写，即图像读取的意思，。

1、 cv2.imread()：读入图片，共两个参数，第一个参数为要读入的图片文件名，第二个参数为如何读取图片，包括 cv2.IMREAD_COLOR：读入一副彩色图片；cv2.IMREAD_GRAYSCALE：以灰度模式读入图片；cv2.IMREAD_UNCHANGED：读入一幅图片，并包括其 alpha 通道。

2、cv2.imshow()：创建一个窗口显示图片，共两个参数，第一个参数表示窗口名字，可以创建多个窗口中，但是每个窗口不能重名；第二个参数是读入的图片。

3、cv2.waitKey()：键盘绑定函数，共一个参数，表示等待毫秒数，将等待特定的几毫秒，看键盘是否有输入，返回值为 ASCII 值。如果其参数为 0，则表示无限期的等待键盘输入。

4、cv2.destroyAllWindows()：删除建立的全部窗口。

5、cv2.destroyWindows()：删除指定的窗口。

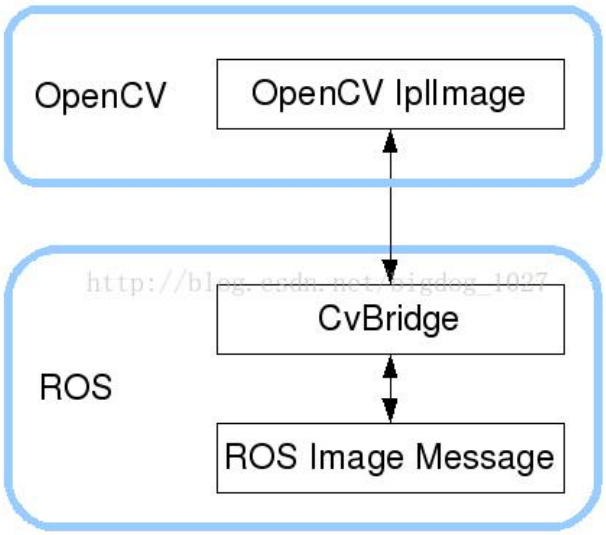
6、cv2.imwrite()：保存图片，共两个参数，第一个为保存文件名，第二个为读入图片。

OpenCV cv_bridge 函数

cv_bridge 函数作用

cv_bridge 用于 ROS 图像和 OpenCV 图像的转换

ROS 对于传感器数据有自己的格式和规范。而在 OpenCV 中，图像是以 Mat 矩阵的形式存储的，这与 ROS 定义的图像消息的格式有一定的区别，所以我们需要利用 cv_bridge 将这两种不相同的格式联系起来，如下图所示



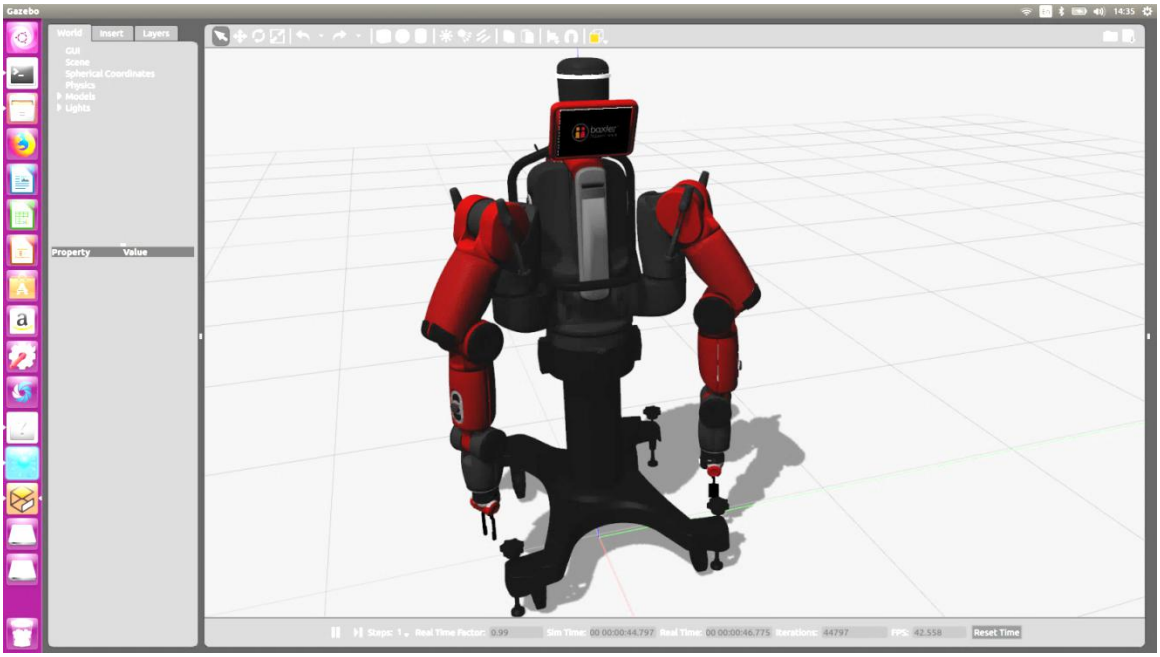
我们使用 imread 和 cv-Bridge 两个函数接口可以完成自定义 Baxter 头部屏幕显示图像的功能。
启动机器人仿真环境
打开终端，运行环境配置脚本:

```
$ source /opt/ros/kinetic/setup.bash
$ cd ~/ros_ws
$    ./baxter.sh    sim
```

通过 launch 文件，启动 gazebo 并载入机器人仿真模型：

```
$ roslaunch baxter_gazebo baxter_world.launch
```

指令成功运行后会出现如下画面，可以看到 baxter 已载入 gazebo 的 3D 视图区，在左侧 world 标签 models 分支下可以查看到 baxter。



（3）运行机器人控制例程

打开新终端，执行以下各条指令：

```
$ cd ~/ros_ws
$    ./baxter.sh    sim
```

在 ros_ws 文件夹中新建文档，输入以下代码并获取文件程序执行权限：

```
#!/usr/bin/python2

import os
import sys
import argparse

import rospy

import cv2
import cv_bridge

from sensor_msgs.msg import (
    Image,
)


def send_image(path):
    """
    Send the image located at the specified path to the head
    display on Baxter.

    @param path: path to the image file to load and send
    """
    img = cv2.imread(path)
    msg = cv_bridge.CvBridge().cv2_to_imgmsg(img, encoding="bgr8")
    pub = rospy.Publisher('/robot/xdisplay', Image, latch=True, queue_size=1)
    pub.publish(msg)
    # Sleep to allow for image to be published.
    rospy.sleep(1)


def main():
    """RSDK Xdisplay Example: Image Display

    Displays a given image file on Baxter's face.

    Pass the relative or absolute file path to an image file on your
    computer, and the example will read and convert the image using
```

```
    cv_bridge, sending it to the screen as a standard ROS Image Message.
    """
    epilog = """
Notes:
    Max screen resolution is 1024x600.
    Images are always aligned to the top-left corner.
    Image formats are those supported by OpenCv - LoadImage().
    """
    arg_fmt = argparse.RawDescriptionHelpFormatter
    parser = argparse.ArgumentParser(formatter_class=arg_fmt,
                                     description=main.__doc__,
                                     epilog=epilog)
    required = parser.add_argument_group('required arguments')
    required.add_argument(
        '-f', '--file', metavar='PATH', required=True,
        help='Path to image file to send'
    )
    parser.add_argument(
        '-d', '--delay', metavar='SEC', type=float, default=0.0,
        help='Time in seconds to wait before publishing image'
    )
    args = parser.parse_args(rospy.myargv()[1:])

    rospy.init_node('rsdk_xdisplay_image', anonymous=True)

    if not os.access(args.file, os.R_OK):
        rospy.logerr("Cannot read file at '%s'" % (args.file,))

        return 1

    # Wait for specified time
    if args.delay > 0:
        rospy.loginfo(
            "Waiting for %s second(s) before publishing image to face" %
            (args.delay,)
        )
        rospy.sleep(args.delay)

    send_image(args.file)
    return 0

if __name__ == '__main__':
    sys.exit(main())
```

程序运行后 Baxter 屏幕上即可显示我们自定义路径下的图片。

五、实验报告

本实验要求完成以下内容：

1.了解 OpenCV 用途

2.在 gazebo 中完成 Baxter 机器人 Xdisplay 程序

3.在 gazebo 中完成 Baxter 屏幕显示头部摄像头画面的程序。

六、注意事项

本实验课为公共科目，由于设备数量有限，请同学们听从实验老师的指导。注意课堂纪律、遵守实验室规章制度和设备使用规范。爱护爱惜实验设备。

湖南瑞森可机器人科技有限公司

官方网址：http://www.cothinkrobotics.com

联系方式：0731-89872400

企业邮箱：info@cothinkrobotics.com

地　　址：湖南省长沙市高新区桐梓坡 348 号国家工程中心